

C# Roadmap — A to Z

25 topics covering C# from basics to .NET reversing & ASP.NET

Generated by BlasterX24 — learn · practice · test

Table of Contents

Topic 01 — Setup & .NET Intro
Topic 02 — Variables & Types
Topic 03 — Operators & Expressions
Topic 04 — Strings
Topic 05 — Arrays & Collections
Topic 06 — Control Flow
Topic 07 — Methods & Params
Topic 08 — OOP: Classes & Objects
Topic 09 — OOP: Inheritance & Interfaces
Topic 10 — OOP: Polymorphism & Generics
Topic 11 — LINQ
Topic 12 — Delegates & Events
Topic 13 — Lambda & Functional
Topic 14 — Exception Handling
Topic 15 — File I/O
Topic 16 — Async/Await
Topic 17 — Regex in C#
Topic 18 — NuGet & Packaging
Topic 19 — ASP.NET Basics
Topic 20 — .NET Reflection
Topic 21 — IL (Intermediate Language)
Topic 22 — .NET Reversing with dnSpy/ILSpy
Topic 23 — Secure Coding in C#
Topic 24 — Memory & Performance
Topic 25 — Capstone Projects

Topic 1: Setup & .NET Intro

C# is a modern, object-oriented language that runs on .NET. You begin by installing the .NET SDK, creating a project with `dotnet new console`, and writing your first `Main` method. The CLI manages build, run, and test workflows so you can focus on code from day one.

Learn From

Start with the official .NET getting-started guide, then explore the CLI reference and the Roslyn compiler docs.

- <https://learn.microsoft.com/en-us/dotnet/core/get-started>
- <https://learn.microsoft.com/en-us/dotnet/core/tools/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/>
- <https://learn.microsoft.com/en-us/dotnet/core/introduction>

Key Concepts

- `dotnet new console` creates a new project with a `Program.cs` and `.csproj` file
- `Main` is the entry point; it can be top-level or a traditional method
- `Console.WriteLine` writes output to the terminal
- `dotnet build` compiles; `dotnet run` builds and executes
- Namespaces organize code; using directives import them
- Project file (`.csproj`) defines target framework, dependencies, and settings
- `dotnet test` runs unit tests from the project
- `dotnet restore` downloads NuGet package dependencies
- Global using directives reduce boilerplate in .NET 6+
- The entry point is discovered by the compiler when no explicit `Main` is defined

Problem 1

What command creates a new C# console project named `MyApp`?

```
dotnet new console -n MyApp
```

Expected output:

```
dotnet new console -n MyApp
```

Learning points:

Problem 2

Write a `Program.cs` that prints `Hello, World!` to the console.

```
Console.WriteLine("Hello, World!");
```

Expected output:

```
Console.WriteLine("Hello, World!");
```

Learning points:

Problem 3

How do you compile and run a project from the project directory?

```
dotnet run
```

Expected output:

```
dotnet run
```

Learning points:

Problem 4

Write a top-level statement Main that prints the current date.

```
Console.WriteLine(DateTime.Now);
```

Expected output:

```
Console.WriteLine(DateTime.Now);
```

Learning points:

Problem 5

What command restores NuGet packages for a project?

```
dotnet restore
```

Expected output:

```
dotnet restore
```

Learning points:

Problem 6

Create a namespace called MyApp.Models and put a class inside it.

```
namespace MyApp.Models { public class User { public string Name { get; set; } } }
```

Expected output:

```
namespace MyApp.Models { public class User { public string Name { get; set; } } }
```

Learning points:

Problem 7

Write code that prints the .NET runtime version to the console.

```
Console.WriteLine(System.Environment.Version);
```

Expected output:

```
Console.WriteLine(System.Environment.Version);
```

Learning points:

Problem 8

What is the extension of a C# project file?

```
.csproj
```

Expected output:

```
.csproj
```

Learning points:

Problem 9

Write a using directive to import the System namespace.

```
using System;
```

Expected output:

```
using System;
```

Learning points:

Problem 10

What command lists installed .NET SDK versions?

```
dotnet --list-sdks
```

Expected output:

```
dotnet --list-sdks
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between dotnet build and dotnet run?

A: dotnet build compiles the project into an assembly but does not execute it; dotnet run compiles and then executes the resulting program.

Q2: Can a C# program have more than one Main method?

A: Only one entry point per project is allowed; the compiler will report an error if multiple Main methods exist without explicit configuration.

Q3: What does the .csproj file control?

A: It defines the target framework, package references, compiler options, output type, and other MSBuild settings for the project.

Q4: What is a top-level statement in C#?

A: It lets you write executable code directly in Program.cs without an explicit Main method or class; the compiler wraps it in a Program class and Main method.

Q5: How do you add a NuGet package from the CLI?

A: Run dotnet add package <PackageName> which adds a PackageReference to the .csproj and restores.

Q6: What is the purpose of global using directives?

A: They let you place a using statement in one file so it is available across all files in the project, reducing repetitive imports.

Q7: What target framework moniker does a new console project use by default?

A: net8.0 (or the latest LTS version installed, such as net6.0).

Q8: What is the role of the Roslyn compiler?

A: Roslyn (Microsoft.CodeAnalysis) is the C# compiler that compiles source code into IL assemblies and provides rich code analysis APIs.

Q9: How do you create a class library project?

A: `dotnet new classlib -n MyLibrary` produces a project that compiles to a DLL instead of an executable.

Q10: What is the difference between console and classlib project templates?

A: Console produces an executable with an entry point; classlib produces a reusable DLL with no entry point.

Topic 2: Variables & Types

C# is statically typed, meaning every variable has a compile-time type. You can use explicit types like `int`, `double`, `string`, and `bool`, or let the compiler infer types with `var`. Constants use `const` and are evaluated at compile time. The `dynamic` keyword defers type checking to runtime for interop scenarios.

Learn From

Review the official C# type reference and the article on implicit typing to understand when to use `var` versus explicit types.

- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-types>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/reference-types>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/var>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/nullable-value-types>

Key Concepts

- Value types (`int`, `double`, `bool`, `char`, `struct`) store data directly on the stack
- Reference types (`string`, `class`, `array`, `delegate`) store a reference on the heap
- `var` lets the compiler infer the type; the variable is still strongly typed
- `const` fields must be assigned at declaration and cannot change
- `readonly` fields can only be assigned in the constructor or field initializer
- `dynamic` disables compile-time type checking; resolved at runtime
- Nullable value types (`int?`) add a null value to value types
- Implicit typing with `var` improves readability when the type is obvious
- `String` is a reference type but is immutable
- Tuple types let you group multiple values without creating a new class

Problem 1

Declare an integer variable named `count` and initialize it to 42.

```
int count = 42;
```

Expected output:

```
int count = 42;
```

Learning points:

Problem 2

Declare a constant string called `APP_NAME` with value `"MyApp"`.

```
const string APP_NAME = "MyApp";
```

Expected output:

```
const string APP_NAME = "MyApp";
```

Learning points:

Problem 3

Use var to declare a variable that holds the value 3.14.

```
var pi = 3.14;
```

Expected output:

```
var pi = 3.14;
```

Learning points:

Problem 4

Declare a nullable int variable named age that can be null.

```
int? age = null;
```

Expected output:

```
int? age = null;
```

Learning points:

Problem 5

Write code that converts a string "100" to an integer.

```
int num = int.Parse("100");
```

Expected output:

```
int num = int.Parse("100");
```

Learning points:

Problem 6

Declare a boolean flag set to true.

```
bool flag = true;
```

Expected output:

```
bool flag = true;
```

Learning points:

Problem 7

Create a tuple with a string name and int age.

```
var person = (Name: "Alice", Age: 30);
```

Expected output:

```
var person = (Name: "Alice", Age: 30);
```

Learning points:

Problem 8

Write code that safely tries to parse a string to int, returning a default on failure.

```
int result = int.TryParse("abc", out int val) ? val : 0;
```

Expected output:

```
int result = int.TryParse("abc", out int val) ? val : 0;
```

Learning points:

Problem 9

Declare a dynamic variable and assign a string to it.

```
dynamic item = "hello";
```

Expected output:

```
dynamic item = "hello";
```

Learning points:

Problem 10

What is the default value of an uninitialized bool field in a class?

```
false
```

Expected output:

```
false
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between const and readonly?

A: const is compile-time constant and implicitly static; readonly is set at runtime (in constructor or field initializer) and is an instance member.

Q2: When should you use var instead of an explicit type?

A: Use var when the type is obvious from the right side of the assignment, such as new expressions or cast results, to improve readability.

Q3: Why is string a reference type but immutable?

A: Strings are reference types allocated on the heap, but once created their contents cannot change; any operation that modifies a string returns a new string instance.

Q4: What happens when you assign null to a non-nullable value type?

A: It causes a compile-time error unless the variable is declared as a nullable value type with the ? suffix.

Q5: What is boxing and unboxing?

A: Boxing wraps a value type in an object on the heap; unboxing extracts the value type from the object, which can throw InvalidCastException if types mismatch.

Q6: What is the difference between string and String?

A: string (lowercase) is an alias for System.String; they compile to the same type, but string is the idiomatic C# keyword.

Q7: Can a const field be of type DateTime?

A: No; const values must be determined at compile time, and DateTime is not a compile-time constant.

Q8: What is dynamic and when would you use it?

A: dynamic bypasses compile-time type checking; use it for COM interop, dynamic objects like ExpandoObject, or reflection-heavy scenarios.

Q9: How do you declare a tuple variable with named elements?

A: var t = (Name: "Bob", Score: 95); then access with t.Name or t.Score.

Q10: What is the difference between var and object?

A: var lets the compiler infer a specific type at compile time; object stores any value but requires casting to use type-specific members.

Topic 3: Operators & Expressions

Operators let you perform calculations, compare values, combine conditions, and work with nulls. C# provides arithmetic (+, -, *, /, %), relational (==, !=, <, >), logical (&&, ||, !), and special operators like null coalescing (??) and pattern matching (is). Understanding operator precedence is essential for writing correct expressions.

Learn From

Study the operator precedence table and practice with pattern matching and null-conditional operators.

- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/pattern-matching>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/null-coalescing-operator>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/member-access-operators>

Key Concepts

- Arithmetic operators (+, -, *, /, %) work on numeric types
- Comparison operators (==, !=, <, >, <=, >=) return bool
- Logical operators (&&, ||, !) combine boolean expressions
- Null coalescing ?? returns the left operand if it is not null, otherwise the right
- Null conditional ?. safely accesses members or indexes on null references
- Pattern matching with is checks types and extracts values
- The ternary operator ?: provides inline if-else expressions
- Compound assignment (+=, -=, *=, /=) combines operation with assignment
- Type-testing patterns include type patterns and constant patterns
- Property patterns let you match on nested properties

Problem 1

Write an expression that returns the remainder of 17 divided by 5.

```
17 % 5
```

Expected output:

```
17 % 5
```

Learning points:

Problem 2

Write a null coalescing expression that assigns name "default" if n is null.

```
string name = n ?? "default";
```

Expected output:

```
string name = n ?? "default";
```

Learning points:

Problem 3

Use the null conditional operator to safely get the length of a possibly null string.

```
int? len = s?.Length;
```

Expected output:

```
int? len = s?.Length;
```

Learning points:

Problem 4

Write a pattern matching expression that checks if obj is an int and prints it.

```
if (obj is int i) Console.WriteLine(i);
```

Expected output:

```
if (obj is int i) Console.WriteLine(i);
```

Learning points:

Problem 5

Write a ternary expression that sets x to 10 if y is greater than 5, else 0.

```
int x = y > 5 ? 10 : 0;
```

Expected output:

```
int x = y > 5 ? 10 : 0;
```

Learning points:

Problem 6

Write a compound assignment that doubles the value of count.

```
count *= 2;
```

Expected output:

```
count *= 2;
```

Learning points:

Problem 7

Write a switch expression that maps true to "Yes" and false to "No".

```
string result = val switch { true => "Yes", false => "No" };
```

Expected output:

```
string result = val switch { true => "Yes", false => "No" };
```

Learning points:

Problem 8

Write an expression that returns the larger of two integers a and b.

```
int max = a > b ? a : b;
```

Expected output:

```
int max = a > b ? a : b;
```

Learning points:

Problem 9

Use a relational pattern to check if a temperature is between 20 and 30 inclusive.

```
if (temp is >= 20 and <= 30) Console.WriteLine("Comfortable");
```

Expected output:

```
if (temp is >= 20 and <= 30) Console.WriteLine("Comfortable");
```

Learning points:

Problem 10

Write code that uses the ??= operator to assign a value only if the variable is null.

```
list ??= new List<string>();
```

Expected output:

```
list ??= new List<string>();
```

Learning points:

Quick Questions & Answers

Q1: What is the precedence order of arithmetic vs comparison vs logical operators?

A: Arithmetic (* / %) have highest precedence, then relational (< > == !=), then logical AND (&&), then logical OR (||), then assignment (= += ??=).

Q2: What does the ??= operator do?

A: It assigns the right-hand value to the left-hand variable only if the variable is currently null.

Q3: How does the null conditional ?. differ from a normal member access?

A: ?. short-circuits; if the left operand is null, the entire expression evaluates to null instead of throwing NullReferenceException.

Q4: Can you use pattern matching to check multiple types in a switch expression?

A: Yes; each arm can use type patterns, constant patterns, or property patterns to match and extract values.

Q5: What is the difference between == and .Equals()?

A: == can be overloaded and may have different behavior for reference types; .Equals() is the virtual method on Object that compares values.

Q6: What does the 'is' operator do in pattern matching?

A: It checks if a value matches a pattern (type, constant, property, or relational) and optionally assigns the matched value to a variable.

Q7: Why does C# use short-circuit evaluation for && and ||?

A: The right side is only evaluated if needed; this prevents unnecessary computation and avoids side effects.

Q8: What is the difference between = and ==?

A: = is the assignment operator; == is the equality comparison operator.

Q9: Can you define custom operators in C#?

A: Yes, you can overload most operators using the operator keyword for user-defined types.

Q10: What does the 'and' keyword do in pattern matching?

A: It combines two patterns; both patterns must match for the combined pattern to succeed.

Topic 4: Strings

Strings in C# are immutable sequences of characters. The `String` class provides methods for searching, splitting, replacing, and formatting. String interpolation (`$""`) lets you embed expressions directly. `StringBuilder` is used for efficient concatenation in loops. Regular expressions via the `Regex` class enable complex pattern matching.

Learn From

Explore the `String` class reference, string interpolation docs, and `StringBuilder` for performance-sensitive concatenation.

- <https://learn.microsoft.com/en-us/dotnet/csharp/how-to/strings-csharp>
- <https://learn.microsoft.com/en-us/dotnet/fundamentals/runtime-libraries/system-string>
- <https://learn.microsoft.com/en-us/dotnet/fundamentals/runtime-libraries/system-text-stringbuilder>
- <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions>

Key Concepts

- Strings are immutable; every modification creates a new string
- String interpolation with `$""` embeds expressions inside `{ }`
- `StringBuilder` is mutable and efficient for building strings in loops
- Common string methods: `Contains`, `StartsWith`, `EndsWith`, `IndexOf`, `Replace`, `Split`, `Trim`
- `string.Join` combines an array/collection into a single string with a separator
- `string.Format` with numbered placeholders `{0}` `{1}` formats output
- Null and empty strings are different; use `string.IsNullOrEmpty()` to check both
- Verbatim strings `@"..."` preserve backslashes and allow multi-line text
- Raw string literals `"""..."""` are new in C# 11 for multi-line raw text
- String comparison uses `StringComparison` enum for culture-aware or ordinal checks

Problem 1

Use string interpolation to print "Hello, Alice! You are 30 years old."

```
string msg = $"Hello, {name}! You are {age} years old.";
```

Expected output:

```
string msg = $"Hello, {name}! You are {age} years old.";
```

Learning points:

Problem 2

Write code that splits a CSV string "a,b,c" into an array.

```
string[] parts = "a,b,c".Split(',');
```

Expected output:

```
string[] parts = "a,b,c".Split(',');
```

Learning points:

Problem 3

Use StringBuilder to build "aaa" by appending "a" three times.

```
var sb = new System.Text.StringBuilder(); for (int i = 0; i < 3; i++) sb.Append("a"); string result = sb.ToS
```

Expected output:

```
var sb = new System.Text.StringBuilder(); for (int i = 0; i < 3; i++) sb.Append("a"); string result = sb.ToString()
```

Learning points:

Problem 4

Check if a string is null or empty.

```
bool empty = string.IsNullOrEmpty(input);
```

Expected output:

```
bool empty = string.IsNullOrEmpty(input);
```

Learning points:

Problem 5

Replace all spaces in a string with underscores.

```
string result = input.Replace(" ", "_");
```

Expected output:

```
string result = input.Replace(" ", "_");
```

Learning points:

Problem 6

Convert a string to lowercase and remove leading/trailing whitespace.

```
string result = input.ToLower().Trim();
```

Expected output:

```
string result = input.ToLower().Trim();
```

Learning points:

Problem 7

Find the index of the first occurrence of "world" in a string.

```
int idx = "hello world".IndexOf("world");
```

Expected output:

```
int idx = "hello world".IndexOf("world");
```

Learning points:

Problem 8

Join an array of strings with a comma separator.

```
string csv = string.Join(",", new[] { "a", "b", "c" });
```

Expected output:

```
string csv = string.Join(",", new[] { "a", "b", "c" });
```

Learning points:

Problem 9

Use a verbatim string to assign a path with backslashes.

```
string path = @"C:\Users\Alice\Documents";
```

Expected output:

```
string path = @"C:\Users\Alice\Documents";
```

Learning points:

Problem 10

Check if a string starts with "http" using case-insensitive comparison.

```
bool ok = url.StartsWith("http", StringComparison.OrdinalIgnoreCase);
```

Expected output:

```
bool ok = url.StartsWith("http", StringComparison.OrdinalIgnoreCase);
```

Learning points:

Quick Questions & Answers

Q1: Why are strings immutable in C#?

A: Immutability ensures thread safety, enables string interning, and simplifies hash code computation for dictionary keys.

Q2: When should you use StringBuilder over concatenation?

A: Use StringBuilder when building strings in loops or with many modifications; each + concatenation creates a new string, while StringBuilder mutates a buffer.

Q3: What is the difference between string.Empty and ""?

A: They are equivalent; both represent a zero-length string. string.Empty is more readable and avoids allocation of a string literal.

Q4: How do you perform a case-insensitive search in a string?

A: Use StringComparison.OrdinalIgnoreCase with methods like Contains, IndexOf, or StartsWith.

Q5: What is string interpolation's performance impact?

A: It is syntactic sugar that the compiler converts to string.Format calls; for tight loops, StringBuilder is still preferred.

Q6: What does the @ prefix do in a string literal?

A: It creates a verbatim string where backslashes are treated as literal characters and line breaks are preserved.

Q7: How do you convert a char array back to a string?

A: Use new string(charArray) or string.Concat(charArray).

Q8: What is the difference between Replace and Substring?

A: Replace swaps all occurrences of a pattern with a replacement; Substring extracts a portion of the string by index and length.

Q9: Can you modify a string in place?

A: No; strings are immutable. You must create a new string or use StringBuilder to build text incrementally.

Q10: What does StringComparison.Ordinal do?

A: It performs a byte-by-byte comparison of the string characters, which is fast but culture-unaware.

Topic 5: Arrays & Collections

Arrays and collections store groups of values. Arrays have fixed size, while `List<T>` and `Dictionary<K,V>` grow dynamically. LINQ provides a declarative way to query, filter, and transform collections using method syntax (`Where`, `Select`, `OrderBy`) or query syntax (`from ... where ... select`).

Learn From

Learn the Array and generic collection classes, then practice with LINQ to manipulate data efficiently.

- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/arrays/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/generics/generic-collections>
- <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>
- <https://learn.microsoft.com/en-us/dotnet/api/system.linq>

Key Concepts

- Arrays are fixed-size, zero-indexed, and can be single or multidimensional
- `List<T>` is a generic, resizable array backed by an internal array
- `Dictionary<K,V>` stores key-value pairs with $O(1)$ lookup
- `Queue<T>` is FIFO; `Stack<T>` is LIFO
- `HashSet<T>` stores unique elements with $O(1)$ contains checks
- LINQ method syntax uses extension methods like `.Where().Select()`
- LINQ query syntax uses `from`, `where`, `orderby`, `select` keywords
- `Array.Copy` and `Array.Resize` manipulate array contents
- Collection initializers let you populate collections inline
- Index and Range operators (`[^1]`, `0..3`) simplify array slicing

Problem 1

Declare and initialize an integer array with values 1, 2, 3.

```
int[] arr = new int[] { 1, 2, 3 };
```

Expected output:

```
int[] arr = new int[] { 1, 2, 3 };
```

Learning points:

Problem 2

Add the number 4 to the end of a `List<int>` named `nums`.

```
nums.Add(4);
```

Expected output:

```
nums.Add(4);
```

Learning points:

Problem 3

Use LINQ to select only even numbers from an int array.

```
var evens = arr.Where(x => x % 2 == 0);
```

Expected output:

```
var evens = arr.Where(x => x % 2 == 0);
```

Learning points:

Problem 4

Use LINQ query syntax to order strings alphabetically.

```
var sorted = from s in names orderby s select s;
```

Expected output:

```
var sorted = from s in names orderby s select s;
```

Learning points:

Problem 5

Get the length of a string array.

```
int len = arr.Length;
```

Expected output:

```
int len = arr.Length;
```

Learning points:

Problem 6

Check if a Dictionary<string,int> contains a specific key.

```
bool exists = dict.ContainsKey("Alice");
```

Expected output:

```
bool exists = dict.ContainsKey("Alice");
```

Learning points:

Problem 7

Use LINQ to get the sum of all elements in an int array.

```
int total = arr.Sum();
```

Expected output:

```
int total = arr.Sum();
```

Learning points:

Problem 8

Convert a `List<int>` to an array.

```
int[] array = nums.ToArray();
```

Expected output:

```
int[] array = nums.ToArray();
```

Learning points:

Problem 9

Use the index operator to get the last element of an array.

```
int last = arr[^1];
```

Expected output:

```
int last = arr[^1];
```

Learning points:

Problem 10

Use the range operator to get the first three elements of an array.

```
int[] first3 = arr[0..3];
```

Expected output:

```
int[] first3 = arr[0..3];
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between an array and `List<T>`?

A: Arrays have fixed size; `List<T>` is a dynamic, resizable collection that internally uses an array and copies elements when capacity is exceeded.

Q2: How does `Dictionary<K,V>` handle hash collisions?

A: It uses chaining (linked lists per bucket) or open addressing depending on the .NET version; collisions degrade lookup from $O(1)$ to $O(n)$ in the worst case.

Q3: What is the difference between `List<T>.Add` and `List<T>.Insert`?

A: `Add` appends to the end; `Insert` places an element at a specific index, shifting subsequent elements.

Q4: How do you iterate over a `Dictionary<string,int>`?

A: Use `foreach` with `KeyValuePair<string,int>`, or iterate over `dict.Keys` or `dict.Values` directly.

Q5: What does LINQ's `.Select()` do?

A: It projects each element into a new form by applying a transform function, returning an `IEnumerable` of the projected type.

Q6: When should you use a `HashSet<T>`?

A: When you need fast $O(1)$ membership checks and want to ensure uniqueness without storing duplicate values.

Q7: What is the difference between `Queue<T>` and `Stack<T>`?

A: `Queue<T>` is first-in-first-out (FIFO); `Stack<T>` is last-in-first-out (LIFO).

Q8: How do you sort an array in place?

A: Use `Array.Sort(arr)` or `arr.Sort()` for a `List<T>`.

Q9: What does the 'from' keyword do in LINQ query syntax?

A: It introduces a range variable that iterates over the source collection, similar to `foreach`.

Q10: How do you remove all elements matching a condition from a `List<T>`?

A: Use `list.RemoveAll(predicate)` which removes all elements for which the predicate returns true.

Topic 6: Control Flow

Control flow determines the order in which statements execute. C# provides conditional statements (if, else, switch), iteration statements (for, foreach, while, do-while), and jump statements (break, continue, return, goto). Pattern matching in switch expressions adds powerful type-based dispatch.

Learn From

Review the control flow statements reference and practice with pattern matching in switch expressions.

- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/jump-statements>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/switch>

Key Concepts

- if/else checks a bool condition and branches execution
- switch matches a value against constant or pattern cases
- switch expressions return a value from pattern-matching arms
- for loops use init; condition; iterator for counted iteration
- foreach iterates over any IEnumerable or IEnumerable<T>
- while evaluates its condition before each iteration
- do-while evaluates its condition after each iteration
- break exits the nearest enclosing loop or switch
- continue skips to the next iteration of the nearest enclosing loop
- throw returns from a method and signals an error

Problem 1

Write an if/else that prints "positive" if $n > 0$, else "non-positive".

```
if (n > 0) Console.WriteLine("positive"); else Console.WriteLine("non-positive");
```

Expected output:

```
if (n > 0) Console.WriteLine("positive"); else Console.WriteLine("non-positive");
```

Learning points:

Problem 2

Write a for loop that prints numbers 1 through 5.

```
for (int i = 1; i <= 5; i++) Console.WriteLine(i);
```

Expected output:

```
for (int i = 1; i <= 5; i++) Console.WriteLine(i);
```

Learning points:

Problem 3

Write a foreach loop that prints each string in a string array.

```
foreach (var s in arr) Console.WriteLine(s);
```

Expected output:

```
foreach (var s in arr) Console.WriteLine(s);
```

Learning points:

Problem 4

Write a while loop that decrements a counter until it reaches 0.

```
while (count > 0) { Console.WriteLine(count); count--; }
```

Expected output:

```
while (count > 0) { Console.WriteLine(count); count--; }
```

Learning points:

Problem 5

Write a switch expression that maps 1 to "One", 2 to "Two", and default to "Other".

```
string result = num switch { 1 => "One", 2 => "Two", _ => "Other" };
```

Expected output:

```
string result = num switch { 1 => "One", 2 => "Two", _ => "Other" };
```

Learning points:

Problem 6

Write code that breaks out of a foreach loop when a negative number is found.

```
foreach (var n in nums) { if (n < 0) break; Console.WriteLine(n); }
```

Expected output:

```
foreach (var n in nums) { if (n < 0) break; Console.WriteLine(n); }
```

Learning points:

Problem 7

Write a do-while loop that asks for input until "quit" is entered.

```
string input; do { input = Console.ReadLine(); } while (input != "quit");
```

Expected output:

```
string input; do { input = Console.ReadLine(); } while (input != "quit");
```

Learning points:

Problem 8

Write an if/else if/else chain that categorizes a score into A, B, C, or F.

```
if (score >= 90) Console.WriteLine("A"); else if (score >= 80) Console.WriteLine("B"); else if (score >= 70) Console.WriteLine("C"); else if (score >= 60) Console.WriteLine("D"); else Console.WriteLine("F");
```

Expected output:

```
if (score >= 90) Console.WriteLine("A"); else if (score >= 80) Console.WriteLine("B"); else if (score >= 70) Console.WriteLine("C"); else if (score >= 60) Console.WriteLine("D"); else Console.WriteLine("F");
```

Learning points:

Problem 9

Write a switch statement that prints the day name for a DayOfWeek enum value.

```
switch (day) { case DayOfWeek.Monday: Console.WriteLine("Monday"); break; case DayOfWeek.Friday: Console.WriteLine("Friday"); break; case DayOfWeek.Saturday: Console.WriteLine("Saturday"); break; case DayOfWeek.Sunday: Console.WriteLine("Sunday"); break; case DayOfWeek.Tuesday: Console.WriteLine("Tuesday"); break; case DayOfWeek.Wednesday: Console.WriteLine("Wednesday"); break; case DayOfWeek.Thursday: Console.WriteLine("Thursday"); break; }
```

Expected output:

```
switch (day) { case DayOfWeek.Monday: Console.WriteLine("Monday"); break; case DayOfWeek.Friday: Console.WriteLine("Friday"); break; case DayOfWeek.Saturday: Console.WriteLine("Saturday"); break; case DayOfWeek.Sunday: Console.WriteLine("Sunday"); break; case DayOfWeek.Tuesday: Console.WriteLine("Tuesday"); break; case DayOfWeek.Wednesday: Console.WriteLine("Wednesday"); break; case DayOfWeek.Thursday: Console.WriteLine("Thursday"); break; }
```

Learning points:

Problem 10

Write a continue statement that skips odd numbers in a for loop from 0 to 9.

```
for (int i = 0; i < 10; i++) { if (i % 2 != 0) continue; Console.WriteLine(i); }
```

Expected output:

```
for (int i = 0; i < 10; i++) { if (i % 2 != 0) continue; Console.WriteLine(i); }
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between switch statement and switch expression?

A: A switch statement executes statements for each case; a switch expression returns a value and uses pattern-matching arms with => syntax.

Q2: Can you use string values in a switch expression?

A: Yes; switch expressions support constant patterns for strings and other types that implement equality comparison.

Q3: What does the _ (discard) mean in a switch expression?

A: It acts as the default case, matching any value not covered by earlier arms.

Q4: When should you use a do-while loop instead of while?

A: When you need the body to execute at least once before checking the condition, such as menu prompts or input validation.

Q5: What is the difference between break and continue?

A: break exits the loop entirely; continue skips to the next iteration without finishing the current iteration.

Q6: Can you nest switch statements?

A: Yes; you can nest any control flow statement inside another, though deep nesting is often better refactored into methods.

Q7: What is a fall-through in switch and does C# allow it?

A: Fall-through means executing a case without a break; C# does not allow it—every case must end with break, return,

throw, or goto.

Q8: What is the pattern matching `_` in if statements?

A: The discard pattern `_` matches any value and is used in switch expressions as the default case.

Q9: Can foreach iterate over any collection type?

A: foreach works on any type that implements `IEnumerable` or `IEnumerable<T>`, or has a `GetEnumerator` method.

Q10: What happens if a for loop condition is always true?

A: The loop runs forever (infinite loop); you must use `break` or `return` to exit it.

Topic 7: Methods & Params

Methods are the building blocks of C# programs. They encapsulate reusable logic and can accept parameters by value, by reference (ref), as output (out), or as read-only (in). Default parameter values let callers omit arguments. The params keyword enables variable-length argument lists. Expression-bodied members provide concise syntax for simple methods.

Learn From

Study the method parameter modifiers reference and practice with ref, out, and params to understand value vs reference semantics.

- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/methods>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/ref>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/out-parameter-modifier>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/params>

Key Concepts

- Methods are defined with access modifier, return type, name, and parameter list
- Passing by value copies the value; ref passes by reference allowing modification
- out parameters must be assigned inside the method before returning
- in parameters are read-only references; cannot be modified
- Default parameter values let callers omit arguments with matching defaults
- params allows a variable number of arguments as an array
- Expression-bodied members use => for one-liner methods and properties
- Local functions are methods defined inside other methods
- Method overloading allows multiple methods with the same name but different signatures
- Static methods belong to the type, not an instance

Problem 1

Write a method Add that takes two ints and returns their sum.

```
int Add(int a, int b) => a + b;
```

Expected output:

```
int Add(int a, int b) => a + b;
```

Learning points:

Problem 2

Write a method DoubleRef that uses ref to double a parameter.

```
void DoubleRef(ref int x) { x *= 2; }
```

Expected output:

```
void DoubleRef(ref int x) { x *= 2; }
```

Learning points:

Problem 3

Write a method TryDivide that returns true via out and the result via a second out.

```
bool TryDivide(int a, int b, out double result) { result = 0; if (b == 0) return false; result = (double)a / b; return true; }
```

Expected output:

```
bool TryDivide(int a, int b, out double result) { result = 0; if (b == 0) return false; result = (double)a / b; return true; }
```

Learning points:

Problem 4

Write a method with a default parameter greeting = "Hello".

```
void Greet(string greeting = "Hello") { Console.WriteLine(greeting); }
```

Expected output:

```
void Greet(string greeting = "Hello") { Console.WriteLine(greeting); }
```

Learning points:

Problem 5

Write a method using params to accept any number of ints and return their sum.

```
int Sum(params int[] numbers) { int total = 0; foreach (var n in numbers) total += n; return total; }
```

Expected output:

```
int Sum(params int[] numbers) { int total = 0; foreach (var n in numbers) total += n; return total; }
```

Learning points:

Problem 6

Write an expression-bodied method that returns the square of an int.

```
int Square(int x) => x * x;
```

Expected output:

```
int Square(int x) => x * x;
```

Learning points:

Problem 7

Write a method that takes an in parameter and prints it without modifying it.

```
void Print(in int value) { Console.WriteLine(value); }
```

Expected output:

```
void Print(in int value) { Console.WriteLine(value); }
```

Learning points:

Problem 8

Overload a method Print to accept either an int or a string.

```
void Print(int n) { Console.WriteLine(n); } void Print(string s) { Console.WriteLine(s); }
```

Expected output:

```
void Print(int n) { Console.WriteLine(n); } void Print(string s) { Console.WriteLine(s); }
```

Learning points:

Problem 9

Write a local function inside a method that calculates factorial.

```
int Factorial(int n) { int fact(int x) => x <= 1 ? 1 : x * fact(x - 1); return fact(n); }
```

Expected output:

```
int Factorial(int n) { int fact(int x) => x <= 1 ? 1 : x * fact(x - 1); return fact(n); }
```

Learning points:

Problem 10

Write a method with named arguments so the caller can pass arguments in any order.

```
void Build(string name, int age) { }
```

Expected output:

```
void Build(string name, int age) { }
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between ref and out parameters?

A: ref requires the variable to be initialized before calling the method; out does not require pre-initialization but must be assigned inside the method.

Q2: Can default parameter values be used with ref or out?

A: No; ref, out, and in parameters cannot have default values because they require an explicit variable reference.

Q3: What is the purpose of the in parameter modifier?

A: It passes a value by reference but prevents modification, useful for large structs to avoid copying while ensuring immutability.

Q4: How does params differ from a regular array parameter?

A: params lets the caller pass individual arguments that the compiler wraps into an array; a regular array parameter requires the caller to construct the array.

Q5: Can you overload methods based on return type alone?

A: No; overloading is based on parameter list (name, count, types, modifiers), not on return type.

Q6: What is an expression-bodied method?

A: A concise syntax using => that replaces the method body with a single expression; the compiler infers the return.

Q7: What are named arguments?

A: Named arguments let you specify the parameter name when calling a method, allowing arguments to be passed in

any order.

Q8: What is the difference between a static method and an instance method?

A: A static method belongs to the type and can be called without an instance; an instance method requires an object and can access instance members.

Q9: What is a local function?

A: A function defined inside another method that can capture variables from the enclosing scope, useful for helper logic.

Q10: Can you use params with named arguments?

A: Yes, but the params array must be the last parameter; named arguments can precede it.

Topic 8: OOP: Classes & Objects

Classes are the fundamental building blocks of object-oriented programming in C#. They encapsulate data (fields) and behavior (methods). Properties provide controlled access to fields. Constructors initialize objects. Static members belong to the type itself. Readonly fields ensure immutability after construction.

Learn From

Review the classes and objects tutorial, then study properties, constructors, and access modifiers in depth.

- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/classes>
- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/properties>
- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors>
-

<https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/static-classes-and-static-class-members>

Key Concepts

- A class is a blueprint for objects; objects are instances of a class
- Fields store data; properties wrap fields with get/set accessors
- Auto-implemented properties { get; set; } generate backing fields automatically
- Constructors are special methods called when creating an object
- Static constructors run once when the type is first used
- Static members belong to the type, not any particular instance
- Readonly fields can only be assigned in the constructor or field initializer
- Access modifiers (public, private, protected, internal) control visibility
- The 'this' keyword refers to the current instance
- Dispose pattern manages unmanaged resources deterministically

Problem 1

Create a Person class with Name and Age auto-properties.

```
class Person { public string Name { get; set; } public int Age { get; set; } }
```

Expected output:

```
class Person { public string Name { get; set; } public int Age { get; set; } }
```

Learning points:

Problem 2

Write a constructor for Person that takes name and age as parameters.

```
class Person { public string Name { get; set; } public int Age { get; set; } public Person(string name, int age) {
```

Expected output:

```
class Person { public string Name { get; set; } public int Age { get; set; } public Person(string name, int age) {
```

Learning points:

Problem 3

Add a readonly field Id to Person that is set in the constructor.

```
class Person { public readonly int Id; public Person(int id) { Id = id; } }
```

Expected output:

```
class Person { public readonly int Id; public Person(int id) { Id = id; } }
```

Learning points:

Problem 4

Create a static class MathHelper with a static method Add.

```
static class MathHelper { public static int Add(int a, int b) => a + b; }
```

Expected output:

```
static class MathHelper { public static int Add(int a, int b) => a + b; }
```

Learning points:

Problem 5

Write a property with only a getter that computes FullName from First and Last.

```
public string FullName => $"{First} {Last}";
```

Expected output:

```
public string FullName => $"{First} {Last}";
```

Learning points:

Problem 6

Create a private field _count and expose it via a public Count property.

```
class Counter { private int _count; public int Count { get { return _count; } set { _count = value; } } }
```

Expected output:

```
class Counter { private int _count; public int Count { get { return _count; } set { _count = value; } } }
```

Learning points:

Problem 7

Write an object initializer to create a Person with Name="Alice" and Age=30.

```
var p = new Person { Name = "Alice", Age = 30 };
```

Expected output:

```
var p = new Person { Name = "Alice", Age = 30 };
```

Learning points:

Problem 8

Create a static constructor that initializes a static field.

```
class Config { public static string AppName; static Config() { AppName = "MyApp"; } }
```

Expected output:

```
class Config { public static string AppName; static Config() { AppName = "MyApp"; } }
```

Learning points:

Problem 9

Use the 'this' keyword to disambiguate a parameter from a field.

```
class Person { string name; public Person(string name) { this.name = name; } }
```

Expected output:

```
class Person { string name; public Person(string name) { this.name = name; } }
```

Learning points:

Problem 10

Add a private setter to the Id property so it can only be set internally.

```
public int Id { get; private set; }
```

Expected output:

```
public int Id { get; private set; }
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between a field and a property?

A: A field is a variable in a class; a property provides get/set accessors that can add validation, logging, or computed logic.

Q2: When is a static constructor called?

A: Exactly once, before any static member is accessed or any instance is created.

Q3: Can a readonly field be modified after construction?

A: No; readonly fields can only be assigned in the constructor or field initializer, not afterward.

Q4: What does 'this' refer to in a class?

A: It refers to the current instance of the class, allowing access to instance members and disambiguation from local variables.

Q5: What is the difference between private and protected access?

A: Private members are accessible only within the declaring class; protected members are also accessible in derived classes.

Q6: What is an object initializer?

A: A syntax that lets you set properties at object creation time without calling a constructor, using { Prop = value } after new.

Q7: Can properties have different access levels for get and set?

A: Yes; you can write { get; private set; } or { private get; set; } to restrict access on one side.

Q8: What is the purpose of the 'new' keyword when hiding a base member?

A: It explicitly indicates you are hiding an inherited member; the compiler warns if you omit 'new' when hiding.

Q9: What is the dispose pattern?

A: A pattern for deterministically releasing unmanaged resources by implementing IDisposable with a Dispose method and optional finalizer.

Q10: What is the difference between a struct and a class?

A: Structs are value types (copied by value); classes are reference types (copied by reference). Structs do not support inheritance.

Topic 9: OOP: Inheritance & Interfaces

Inheritance lets a class derive from a base class, inheriting its members. Abstract classes define incomplete contracts that derived classes must fulfill. Interfaces define a contract of members that implementing classes must provide. Together they enable polymorphism, code reuse, and clean architectural separation.

Learn From

Study inheritance and interface implementation, focusing on virtual/override, abstract classes, and interface default implementations.

- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/interface>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual>

Key Concepts

- A derived class inherits all non-private members from its base class
- virtual members can be overridden in derived classes with override
- abstract members have no implementation and must be overridden
- sealed classes prevent further inheritance
- An interface defines a contract of methods, properties, events, or indexers
- A class can implement multiple interfaces but inherit from only one class
- The 'base' keyword accesses members of the parent class
- Interface default implementations provide fallback code in C# 8+
- Explicit interface implementations resolve ambiguity between multiple interfaces
- The Liskov Substitution Principle requires derived classes to be substitutable for base

Problem 1

Create a base class Animal with a virtual Speak method.

```
class Animal { public virtual void Speak() { Console.WriteLine("..."); } }
```

Expected output:

```
class Animal { public virtual void Speak() { Console.WriteLine("..."); } }
```

Learning points:

Problem 2

Create a Dog class that overrides Animal.Speak to print "Woof!".

```
class Dog : Animal { public override void Speak() { Console.WriteLine("Woof!"); } }
```

Expected output:

```
class Dog : Animal { public override void Speak() { Console.WriteLine("Woof!"); } }
```

Learning points:

Problem 3

Define an interface ISwim with a Swim method.

```
interface ISwim { void Swim(); }
```

Expected output:

```
interface ISwim { void Swim(); }
```

Learning points:

Problem 4

Make Duck inherit from Animal and implement ISwim.

```
class Duck : Animal, ISwim { public void Swim() { Console.WriteLine("Swimming"); } }
```

Expected output:

```
class Duck : Animal, ISwim { public void Swim() { Console.WriteLine("Swimming"); } }
```

Learning points:

Problem 5

Create an abstract class Shape with an abstract Area method.

```
abstract class Shape { public abstract double Area(); }
```

Expected output:

```
abstract class Shape { public abstract double Area(); }
```

Learning points:

Problem 6

Create a Circle class that inherits Shape and implements Area.

```
class Circle : Shape { double radius; public Circle(double r) { radius = r; } public override double Area() { return Math.PI * radius * radius; } }
```

Expected output:

```
class Circle : Shape { double radius; public Circle(double r) { radius = r; } public override double Area() => Math.PI * radius * radius; }
```

Learning points:

Problem 7

Use base to call the parent constructor from a derived class.

```
class Derived : Base { public Derived(int x) : base(x) { } }
```

Expected output:

```
class Derived : Base { public Derived(int x) : base(x) { } }
```

Learning points:

Problem 8

Seal a class so it cannot be inherited.

```
sealed class FinalClass { }
```

Expected output:

```
sealed class FinalClass { }
```

Learning points:

Problem 9

Use an interface to require a Calculate method on any implementing class.

```
interface ICalculator { double Calculate(double a, double b); }
```

Expected output:

```
interface ICalculator { double Calculate(double a, double b); }
```

Learning points:

Problem 10

Explicitly implement an interface method so it is only accessible via the interface type.

```
class MyClass : IMyInterface { void IMyInterface.DoWork() { } }
```

Expected output:

```
class MyClass : IMyInterface { void IMyInterface.DoWork() { } }
```

Learning points:

Quick Questions & Answers

Q1: Why can't C# classes inherit from multiple classes?

A: C# uses single inheritance for classes to avoid the diamond problem; multiple interface implementation provides the flexibility of multiple contracts.

Q2: What is the difference between abstract and virtual?

A: Abstract has no implementation and forces overriding; virtual provides a default implementation that derived classes may optionally override.

Q3: Can you create an instance of an abstract class?

A: No; abstract classes cannot be instantiated directly; they must be subclassed and their abstract members implemented.

Q4: What happens if a class does not implement all interface members?

A: It must be declared abstract, or the compiler reports an error for unimplemented members.

Q5: What is the sealed keyword used for?

A: It prevents a class from being inherited or a virtual/override member from being further overridden.

Q6: What is explicit interface implementation?

A: It implements an interface member with a fully qualified name, making it accessible only through the interface type, not through the class.

Q7: Can an interface have fields?

A: No; interfaces can only declare methods, properties, events, and indexers (C# 11 adds default interface methods).

Q8: What is the 'base' keyword?

A: It accesses the base class's implementation of a method, constructor, or property, useful for extending rather than replacing behavior.

Q9: Can a class inherit an interface and provide a default implementation?

A: Yes; since C# 8, interfaces can contain default implementations that classes inherit unless overridden.

Q10: What is the Liskov Substitution Principle?

A: Objects of a derived class should be substitutable for objects of the base class without altering correctness.

Topic 10: OOP: Polymorphism & Generics

Polymorphism lets you treat derived types as their base type, with the correct behavior dispatched at runtime. Override replaces base behavior, new hides it, and sealed prevents further overriding. Generics let you write type-safe, reusable code that works with any type while avoiding boxing and casts.

Learn From

Study runtime polymorphism (virtual/override), compile-time hiding (new), and generic type constraints.

- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/polymorphism>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/new-modifier>
- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/generics/generic-type-parameters>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/where-clause>

Key Concepts

- Polymorphism allows a base reference to hold a derived object and invoke the correct method
- override replaces the base implementation; the runtime determines which method to call
- new explicitly hides an inherited member without polymorphic dispatch
- sealed override prevents further overriding of a virtual member in a derived class
- Generics enable type-safe, reusable code without boxing or casting
- Generic type constraints (where T : class, struct, new()) restrict what types can be used
- Generic methods can be called with explicit type arguments or inferred types
- Covariance (out) and contravariance (in) allow flexibility in generic type assignment
- A generic class can have multiple type parameters
- Open generics allow runtime type construction while closed generics are fully specified

Problem 1

Write a base class Animal with virtual Speak and a Dog that overrides it.

```
class Animal { public virtual void Speak() { } } class Dog : Animal { public override void Speak() { Console.
```

Expected output:

```
class Animal { public virtual void Speak() { } } class Dog : Animal { public override void Speak() { Console.Writel
```

Learning points:

Problem 2

Use new to hide a base method in a derived class.

```
class Base { public void Do() { } } class Derived : Base { public new void Do() { } }
```

Expected output:

```
class Base { public void Do() { } } class Derived : Base { public new void Do() { } }
```

Learning points:

Problem 3

Create a generic List<T> and add three integers to it.

```
var list = new List<int> { 1, 2, 3 };
```

Expected output:

```
var list = new List<int> { 1, 2, 3 };
```

Learning points:

Problem 4

Write a generic method Max<T> that returns the larger of two values, constrained to IComparable<T>.

```
T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) > 0 ? a : b;
```

Expected output:

```
T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) > 0 ? a : b;
```

Learning points:

Problem 5

Seal an override so no further subclass can override it.

```
class Base { public virtual void Do() { } } class Middle : Base { public sealed override void Do() { } }
```

Expected output:

```
class Base { public virtual void Do() { } } class Middle : Base { public sealed override void Do() { } }
```

Learning points:

Problem 6

Create a generic Pair<T> class with First and Second properties.

```
class Pair<T> { public T First { get; set; } public T Second { get; set; } }
```

Expected output:

```
class Pair<T> { public T First { get; set; } public T Second { get; set; } }
```

Learning points:

Problem 7

Use a generic constraint to require that T is a reference type with a parameterless constructor.

```
class Factory<T> where T : class, new() { public T Create() => new T(); }
```

Expected output:

```
class Factory<T> where T : class, new() { public T Create() => new T(); }
```

Learning points:

Problem 8

Demonstrate covariance using `IEnumerable<out T>`.

```
IEnumerable<Derived> derived = new List<Derived>(); IEnumerable<Base> bases = derived;
```

Expected output:

```
IEnumerable<Derived> derived = new List<Derived>(); IEnumerable<Base> bases = derived;
```

Learning points:

Problem 9

Write a generic `Repository<T>` with a `GetById` method.

```
class Repository<T> where T : class { public T GetById(int id) { return default; } }
```

Expected output:

```
class Repository<T> where T : class { public T GetById(int id) { return default; } }
```

Learning points:

Problem 10

Use `typeof` to get the `Type` of a generic parameter at runtime.

```
Type t = typeof(T);
```

Expected output:

```
Type t = typeof(T);
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between `override` and `new`?

A: `Override` replaces the virtual method and is called polymorphically; `new` hides the base member and is called based on the compile-time type, not the runtime type.

Q2: When should you use `sealed override`?

A: When you want to prevent further derived classes from overriding a virtual method, sealing the implementation.

Q3: What are generic type constraints?

A: They restrict which types can be used as generic arguments using `where` clauses like `class`, `struct`, `new()`, or interface constraints.

Q4: Why are generics better than using `object`?

A: Generics are type-safe at compile time, avoid boxing/unboxing overhead, and eliminate the need for explicit casts.

Q5: What is covariance in generics?

A: Covariance (`out`) allows an interface or delegate to return a more derived type; `IEnumerable<Derived>` can be assigned to `IEnumerable<Base>`.

Q6: What is contravariance in generics?

A: Contravariance (`in`) allows a parameter to accept a less derived type; `Action<Base>` can be assigned to `Action<Derived>`.

Q7: Can a generic class implement an interface?

A: Yes; for example, `List<T>` implements `IEnumerable<T>`, allowing type-safe iteration over any element type.

Q8: What is an open generic type?

A: A generic type with unresolved type parameters, such as `List<T>`; it becomes a closed generic when fully specified as `List<int>`.

Q9: How do you call a generic method with explicit type arguments?

A: Use the syntax `Method<Type>(args)`, e.g., `Max<int>(1, 2)`.

Q10: What is the `default(T)` expression?

A: It returns the default value for a generic type: null for reference types, 0 for numeric types, false for bool, etc.

Topic 11: LINQ

LINQ (Language Integrated Query) provides a unified syntax for querying data from any source—arrays, lists, databases, XML. It offers two styles: method syntax (fluent) using extension methods like `Where`, `Select`, `OrderBy`, and query syntax using `from`, `where`, `orderby`, `select` keywords. Both compile to the same IL.

Learn From

Start with the LINQ overview, then practice converting between method and query syntax.

- <https://learn.microsoft.com/en-us/dotnet/linq/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/linq/write-linq-queries>
- <https://learn.microsoft.com/en-us/dotnet/csharp/linq/standard-query-operators>
- <https://learn.microsoft.com/en-us/dotnet/csharp/linq/method-syntax-vs-query-syntax>

Key Concepts

- `Where` filters elements based on a predicate
- `Select` projects elements into a new form
- `OrderBy` and `OrderByDescending` sort elements
- `GroupBy` groups elements by a key
- `Sum`, `Count`, `Average` aggregate numeric values
- `First`, `Last`, `Single` retrieve individual elements
- `Any` and `All` test conditions across collections
- `Distinct` removes duplicates
- `Join` pairs elements from two sequences based on matching keys
- Deferred execution means queries are not evaluated until enumerated

Problem 1

Use LINQ to filter numbers greater than 5 from an array.

```
var result = arr.Where(x => x > 5);
```

Expected output:

```
var result = arr.Where(x => x > 5);
```

Learning points:

Problem 2

Use `Select` to square each number in an int array.

```
var squares = arr.Select(x => x * x);
```

Expected output:

```
var squares = arr.Select(x => x * x);
```

Learning points:

Problem 3

Order a string array alphabetically using LINQ.

```
var sorted = arr.OrderBy(s => s);
```

Expected output:

```
var sorted = arr.OrderBy(s => s);
```

Learning points:

Problem 4

Group numbers by whether they are even or odd.

```
var groups = arr.GroupBy(x => x % 2 == 0 ? "Even" : "Odd");
```

Expected output:

```
var groups = arr.GroupBy(x => x % 2 == 0 ? "Even" : "Odd");
```

Learning points:

Problem 5

Count how many strings in an array start with "a".

```
int count = arr.Count(s => s.StartsWith("a"));
```

Expected output:

```
int count = arr.Count(s => s.StartsWith("a"));
```

Learning points:

Problem 6

Use LINQ query syntax to select names longer than 3 characters.

```
var result = from n in names where n.Length > 3 select n;
```

Expected output:

```
var result = from n in names where n.Length > 3 select n;
```

Learning points:

Problem 7

Use First to get the first element greater than 10.

```
int first = arr.First(x => x > 10);
```

Expected output:

```
int first = arr.First(x => x > 10);
```

Learning points:

Problem 8

Use Any to check if any element is negative.

```
bool hasNegative = arr.Any(x => x < 0);
```

Expected output:

```
bool hasNegative = arr.Any(x => x < 0);
```

Learning points:

Problem 9

Use Sum to total all elements in an int array.

```
int total = arr.Sum();
```

Expected output:

```
int total = arr.Sum();
```

Learning points:

Problem 10

Use Distinct to remove duplicate integers from a list.

```
var unique = list.Distinct();
```

Expected output:

```
var unique = list.Distinct();
```

Learning points:

Quick Questions & Answers

Q1: What is deferred execution in LINQ?

A: The query is not evaluated when defined; it is executed when enumerated (e.g., in a foreach), allowing lazy evaluation.

Q2: What forces immediate execution?

A: Methods like ToList(), ToArray(), Count(), First(), and Sum() force the query to execute immediately.

Q3: What is the difference between First and FirstOrDefault?

A: First throws InvalidOperationException if no element matches; FirstOrDefault returns default(T) instead.

Q4: How does GroupBy work?

A: It groups elements by a key selector and returns an IEnumerable<IGrouping<TKey, TElement>> where each group has a Key and elements.

Q5: What is the difference between method syntax and query syntax?

A: Method syntax uses extension methods (Where, Select); query syntax uses SQL-like keywords (from, where, select). They compile to the same code.

Q6: How do you flatten a collection of collections?

A: Use SelectMany or query syntax with multiple from clauses.

Q7: What does the AsEnumerable() method do?

A: It returns the source as IEnumerable<T>, switching to LINQ to Objects from a more specific provider like

IQueryable<T>.

Q8: Can you chain multiple LINQ operators?

A: Yes; LINQ operators return IEnumerable<T>, so you can chain Where, Select, OrderBy, etc. in a fluent pipeline.

Q9: What is the difference between All and Any?

A: All returns true if every element satisfies the predicate; Any returns true if at least one element satisfies it.

Q10: How do you get the maximum value with LINQ?

A: Use arr.Max() or arr.Max(x => x.Property) to find the maximum element or maximum projected value.

Topic 12: Delegates & Events

Delegates are type-safe function pointers that reference methods. Action and Func are built-in generic delegates for common signatures. Events use delegates to implement the publish-subscribe pattern, allowing objects to notify subscribers when something happens. Multicast delegates support multiple method targets.

Learn From

Study the delegate type, Action/Func signatures, and the event keyword for implementing observers.

- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/>
- <https://learn.microsoft.com/en-us/dotnet/api/system.action>
- <https://learn.microsoft.com/en-us/dotnet/api/system.func-1>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/event>

Key Concepts

- A delegate defines a method signature; any method matching the signature can be assigned
- Action<T> is a delegate that returns void with up to 16 parameters
- Func<T, TResult> is a delegate that returns a value with up to 16 parameters
- Events are delegates that can only be invoked from the declaring class
- The += operator subscribes to an event; -= unsubscribes
- Multicast delegates chain multiple method calls into one delegate instance
- EventHandler<T> is a standard delegate for .NET events
- Delegate.Combine and Delegate.Remove manage multicast chains
- Anonymous methods and lambdas simplify delegate instantiation
- Delegates enable callback patterns, strategy patterns, and LINQ queries

Problem 1

Declare a delegate int MathOp(int a, int b).

```
delegate int MathOp(int a, int b);
```

Expected output:

```
delegate int MathOp(int a, int b);
```

Learning points:

Problem 2

Assign a lambda to Action<string> that prints a message.

```
Action<string> print = msg => Console.WriteLine(msg);
```

Expected output:

```
Action<string> print = msg => Console.WriteLine(msg);
```

Learning points:

Problem 3

Create a `Func<int, int, int>` that returns the sum of two ints.

```
Func<int, int, int> add = (a, b) => a + b;
```

Expected output:

```
Func<int, int, int> add = (a, b) => a + b;
```

Learning points:

Problem 4

Define an event called `OnClick` using `EventHandler`.

```
public event EventHandler OnClick;
```

Expected output:

```
public event EventHandler OnClick;
```

Learning points:

Problem 5

Invoke an event safely by checking for null first.

```
OnClick?.Invoke(this, EventArgs.Empty);
```

Expected output:

```
OnClick?.Invoke(this, EventArgs.Empty);
```

Learning points:

Problem 6

Subscribe a method to an event using `+=`.

```
obj.OnClick += MyHandler;
```

Expected output:

```
obj.OnClick += MyHandler;
```

Learning points:

Problem 7

Create a multicast delegate by combining two `Action` delegates.

```
Action a = () => Console.WriteLine("A"); Action b = () => Console.WriteLine("B"); Action combined = a + b;
```

Expected output:

```
Action a = () => Console.WriteLine("A"); Action b = () => Console.WriteLine("B"); Action combined = a + b;
```

Learning points:

Problem 8

Pass a `Func<int, bool>` to a `Filter` method that returns matching elements.

```
IEnumerable<int> Filter(IEnumerable<int> source, Func<int, bool> predicate) => source.Where(predicate);
```

Expected output:

```
IEnumerable<int> Filter(IEnumerable<int> source, Func<int, bool> predicate) => source.Where(predicate);
```

Learning points:

Problem 9

Declare an event using `EventHandler<T>` with a custom `EventArgs`.

```
public event EventHandler<ChangedEventArgs> Changed;
```

Expected output:

```
public event EventHandler<ChangedEventArgs> Changed;
```

Learning points:

Problem 10

Unsubscribe from an event using `-=`.

```
obj.OnClick -= MyHandler;
```

Expected output:

```
obj.OnClick -= MyHandler;
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between a delegate and an event?

A: A delegate can be invoked, reassigned, and combined from anywhere; an event can only be invoked from the declaring class and subscribers can only `+=` or `-=`.

Q2: Why use `EventHandler` instead of a custom delegate?

A: `EventHandler` is a standard .NET convention; it provides a consistent signature and works with `EventArgs` for event data.

Q3: What is a multicast delegate?

A: A delegate that holds references to multiple methods; invoking it calls all methods in order.

Q4: When should you use `Action` vs `Func`?

A: Use `Action` when the method returns void; use `Func` when it returns a value.

Q5: What happens if you invoke a delegate that references a disposed object?

A: It depends on the delegate chain; if one target throws, subsequent targets may not be called.

Q6: How do you prevent null delegate invocations?

A: Use the null-conditional operator: `myDelegate?.Invoke(args);`

Q7: Can events be invoked outside the declaring class?

A: No; the event keyword restricts invocation to the class that declared it; external code can only subscribe or unsubscribe.

Q8: What is the purpose of EventArgs?

A: It provides a base class for passing event-specific data to event handlers.

Q9: How do you make a delegate from a static method?

A: Assign the method name directly: `Action print = MyClass.Print;`

Q10: Can a delegate reference multiple methods?

A: Yes; using multicast delegates with `+` or `+=`, you chain multiple methods into one delegate.

Topic 13: Lambda & Functional

Lambda expressions are anonymous functions that can capture variables from their enclosing scope (closures). They are the foundation of LINQ queries, delegate instantiation, and functional programming patterns in C#. Lambdas can be assigned to delegates, passed as arguments, and used in expression trees.

Learn From

Study lambda syntax, closures, and how lambdas integrate with delegates and LINQ.

- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-expressions>
- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/statements-expressions-operators/lambda-expressions>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-operator>
- <https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/expression-trees/>

Key Concepts

- Lambda expressions use `=>` to define anonymous methods
- Single-line lambdas can omit braces and the return keyword
- Multi-line lambdas use braces and an explicit return statement
- Lambdas capture variables from the enclosing scope (closures)
- Captured variables are shared across all invocations of the lambda
- Lambdas can be converted to delegates or expression trees
- Expression trees store the lambda as data, enabling translation to SQL or other languages
- Local variables captured by lambdas live as long as the delegate
- Lambdas enable functional patterns like map, filter, and reduce via LINQ
- Discard parameters `_` let you ignore unused lambda arguments

Problem 1

Write a lambda that takes an int and returns its square.

```
Func<int, int> square = x => x * x;
```

Expected output:

```
Func<int, int> square = x => x * x;
```

Learning points:

Problem 2

Write a lambda that captures a variable and adds it to each element.

```
int addend = 10; Func<int, int> add = n => n + addend;
```

Expected output:

```
int addend = 10; Func<int, int> add = n => n + addend;
```

Learning points:

Problem 3

Use a lambda with Select to convert strings to uppercase.

```
var upper = names.Select(n => n.ToUpper());
```

Expected output:

```
var upper = names.Select(n => n.ToUpper());
```

Learning points:

Problem 4

Write a multi-line lambda that calculates factorial.

```
Func<int, int> fact = n => { int r = 1; for (int i = 2; i <= n; i++) r *= i; return r; };
```

Expected output:

```
Func<int, int> fact = n => { int r = 1; for (int i = 2; i <= n; i++) r *= i; return r; };
```

Learning points:

Problem 5

Use a lambda as a predicate in Where to filter odd numbers.

```
var odds = nums.Where(x => x % 2 != 0);
```

Expected output:

```
var odds = nums.Where(x => x % 2 != 0);
```

Learning points:

Problem 6

Pass a lambda to Array.Find to find the first negative number.

```
int first = Array.Find(arr, x => x < 0);
```

Expected output:

```
int first = Array.Find(arr, x => x < 0);
```

Learning points:

Problem 7

Use a lambda with OrderBy to sort by string length.

```
var sorted = names.OrderBy(n => n.Length);
```

Expected output:

```
var sorted = names.OrderBy(n => n.Length);
```

Learning points:

Problem 8

Write a lambda that returns a tuple with the input and its square.

```
Func<int, (int, int)> pair = x => (x, x * x);
```

Expected output:

```
Func<int, (int, int)> pair = x => (x, x * x);
```

Learning points:

Problem 9

Use a discard in a lambda to ignore the second parameter.

```
Func<int, int, int> first = (a, _) => a;
```

Expected output:

```
Func<int, int, int> first = (a, _) => a;
```

Learning points:

Problem 10

Create a `Func<string, bool>` lambda that checks if a string contains "test".

```
Func<string, bool> containsTest = s => s.Contains("test");
```

Expected output:

```
Func<string, bool> containsTest = s => s.Contains("test");
```

Learning points:

Quick Questions & Answers

Q1: What is a closure in C#?

A: A closure is a lambda that captures variables from its enclosing scope; the captured variables are shared across calls.

Q2: Why do captured variables live longer than the enclosing method?

A: The compiler moves captured variables to a compiler-generated class on the heap, so they survive as long as the delegate referencing them.

Q3: What is the difference between expression lambdas and statement lambdas?

A: Expression lambdas (`x => x + 1`) are single expressions; statement lambdas (`{ return x + 1; }`) use braces and can have multiple statements.

Q4: Can lambdas be converted to expression trees?

A: Yes; assigning a lambda to `Expression<Func<T>>` stores it as a data structure, enabling translation to SQL or other query languages.

Q5: What is a common pitfall with closures in loops?

A: Capturing a loop variable in a lambda captures the variable itself, not its value; all lambdas share the final value unless you create a local copy.

Q6: When should you use a statement lambda over an expression lambda?

A: When the logic requires multiple statements, local variables, or complex control flow that cannot be expressed in a single expression.

Q7: What is a Func delegate?

A: A built-in generic delegate that represents a method with parameters and a return value, e.g., `Func<int, string>`.

Q8: What is an Action delegate?

A: A built-in generic delegate for methods that return void, e.g., `Action<string>` or `Action<int, int>`.

Q9: How does LINQ use lambdas?

A: LINQ operators like `Where`, `Select`, and `OrderBy` accept lambda expressions as predicates or projections, enabling declarative queries.

Q10: Can a lambda capture a ref or out parameter?

A: No; lambdas cannot capture ref or out parameters by design; you must assign the value to a local variable first.

Topic 14: Exception Handling

Exception handling lets you catch and respond to runtime errors gracefully. The try/catch/finally pattern isolates code that might fail, catches specific exceptions, and guarantees cleanup in finally blocks. Custom exceptions carry domain-specific information. The using statement ensures deterministic disposal of resources.

Learn From

Study the exception handling reference, best practices for catching exceptions, and the using declaration.

- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/try-catch-finally>
- <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/creating-and-throwing-exceptions>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/using-statement>

Key Concepts

- try wraps code that might throw an exception
- catch handles specific exception types; multiple catch blocks can handle different types
- finally executes regardless of whether an exception was thrown or caught
- throw raises an exception; throw preserves the original stack trace in catch
- Exception.Message contains the error description; Exception.StackTrace has the call stack
- Custom exceptions inherit from Exception and carry domain-specific data
- using ensures IDisposable objects are disposed even if an exception occurs
- using declaration (C# 8+) uses a scoped using without braces
- NullReferenceException, ArgumentException, and InvalidOperationException are common exceptions
- Exception filters with when let you add conditions to catch blocks

Problem 1

Write a try/catch that catches DivideByZeroException and prints the error.

```
try { int r = 10 / 0; } catch (DivideByZeroException ex) { Console.WriteLine(ex.Message); }
```

Expected output:

```
try { int r = 10 / 0; } catch (DivideByZeroException ex) { Console.WriteLine(ex.Message); }
```

Learning points:

Problem 2

Write a try/finally block that closes a file in the finally.

```
try { /* work */ } finally { File.Close(); }
```

Expected output:

```
try { /* work */ } finally { File.Close(); }
```

Learning points:

Problem 3

Throw an `ArgumentException` with a message.

```
throw new ArgumentException("Invalid name", nameof(name));
```

Expected output:

```
throw new ArgumentException("Invalid name", nameof(name));
```

Learning points:

Problem 4

Create a custom exception class called `OrderNotFoundException`.

```
class OrderNotFoundException : Exception { public OrderNotFoundException(string msg) : base(msg) { } }
```

Expected output:

```
class OrderNotFoundException : Exception { public OrderNotFoundException(string msg) : base(msg) { } }
```

Learning points:

Problem 5

Use a `using` statement to read all text from a file.

```
using (var sr = new StreamReader(path)) { string content = sr.ReadToEnd(); }
```

Expected output:

```
using (var sr = new StreamReader(path)) { string content = sr.ReadToEnd(); }
```

Learning points:

Problem 6

Use a `using` declaration (C# 8+) for a file stream.

```
using var fs = File.OpenRead(path);
```

Expected output:

```
using var fs = File.OpenRead(path);
```

Learning points:

Problem 7

Catch multiple exception types in order from most to least specific.

```
try { /* code */ } catch (InvalidOperationException ex) { } catch (Exception ex) { }
```

Expected output:

```
try { /* code */ } catch (InvalidOperationException ex) { } catch (Exception ex) { }
```

Learning points:

Problem 8

Use a when filter to catch only exceptions with a specific HRESULT.

```
catch (COMException ex) when (ex.ErrorCode == -2147024809) { }
```

Expected output:

```
catch (COMException ex) when (ex.ErrorCode == -2147024809) { }
```

Learning points:

Problem 9

Rethrow an exception preserving the original stack trace.

```
catch (Exception) { throw; }
```

Expected output:

```
catch (Exception) { throw; }
```

Learning points:

Problem 10

Use try/catch with async code and await.

```
try { await Task.Run(() => RiskyOperation()); } catch (Exception ex) { Console.WriteLine(ex.Message); }
```

Expected output:

```
try { await Task.Run(() => RiskyOperation()); } catch (Exception ex) { Console.WriteLine(ex.Message); }
```

Learning points:

Quick Questions & Answers

Q1: Why should you catch specific exceptions instead of Exception?

A: Catching specific exceptions prevents accidentally handling unrelated errors and makes debugging easier.

Q2: What is the difference between throw and throw ex?

A: throw preserves the original stack trace; throw ex resets the stack trace to the current location, losing debugging information.

Q3: What does the finally block guarantee?

A: It executes whether or not an exception was thrown or caught, making it ideal for cleanup code.

Q4: When should you create a custom exception?

A: When you need to convey domain-specific error information that standard exceptions do not carry.

Q5: What is the using statement?

A: It ensures that an IDisposable object is disposed when the block exits, even if an exception occurs.

Q6: Can you have multiple catch blocks?

A: Yes; you can have multiple catch blocks, each handling a different exception type, ordered from most specific to most general.

Q7: What is Exception.InnerException?

A: It provides access to the original exception that caused the current exception, forming an exception chain.

Q8: What is the difference between Exception.Data and Exception.Message?

A: Message is a string description of the error; Data is an IDictionary for arbitrary key-value pairs of diagnostic information.

Q9: Why is catching exceptions control flow considered bad practice?

A: Exceptions are expensive and should signal unexpected errors; use conditional checks (if) for expected conditions.

Q10: How do you implement IDisposable with using?

A: Implement a Dispose method that cleans up resources; using calls Dispose when the block exits.

Topic 15: File I/O

File I/O in C# uses the System.IO namespace to read, write, and manipulate files and directories. The File and Directory static classes provide convenience methods. StreamReader and StreamWriter handle text streams. BinaryReader and BinaryWriter handle binary data. Async methods improve responsiveness for large files.

Learn From

Study the System.IO namespace, focusing on File, Directory, StreamReader, and async file operations.

- <https://learn.microsoft.com/en-us/dotnet/csharp/file-io/>
- <https://learn.microsoft.com/en-us/dotnet/api/system.io.file>
- <https://learn.microsoft.com/en-us/dotnet/api/system.io.streamreader>
- <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/asynchronous-file-io>

Key Concepts

- File.ReadAllText and File.WriteAllText read/write entire files in one call
- File.ReadAllLines and File.WriteAllLines handle line-by-line I/O
- Directory.CreateDirectory creates a directory path
- Directory.GetFiles and Directory.GetDirectories list directory contents
- StreamReader reads text line-by-line or in chunks
- StreamWriter writes text with automatic buffer flushing
- BinaryReader and BinaryWriter handle primitive type serialization
- FileStream provides low-level byte-level access with buffering
- Async methods (ReadAllTextAsync, CopyToAsync) improve UI responsiveness
- File.Exists and Directory.Exists check for existence before operations

Problem 1

Read all text from a file named data.txt.

```
string content = File.ReadAllText("data.txt");
```

Expected output:

```
string content = File.ReadAllText("data.txt");
```

Learning points:

Problem 2

Write "Hello" to a file named output.txt.

```
File.WriteAllText("output.txt", "Hello");
```

Expected output:

```
File.WriteAllText("output.txt", "Hello");
```

Learning points:

Problem 3

Read all lines from a file into a string array.

```
string[] lines = File.ReadAllLines("data.txt");
```

Expected output:

```
string[] lines = File.ReadAllLines("data.txt");
```

Learning points:

Problem 4

Create a directory named reports if it does not exist.

```
Directory.CreateDirectory("reports");
```

Expected output:

```
Directory.CreateDirectory("reports");
```

Learning points:

Problem 5

Use StreamReader to read a file line-by-line.

```
using var sr = new StreamReader("data.txt"); while (sr.Peek() >= 0) { string line = sr.ReadLine(); }
```

Expected output:

```
using var sr = new StreamReader("data.txt"); while (sr.Peek() >= 0) { string line = sr.ReadLine(); }
```

Learning points:

Problem 6

Use StreamWriter to append text to a file.

```
using var sw = new StreamWriter("log.txt", true); sw.WriteLine("New entry");
```

Expected output:

```
using var sw = new StreamWriter("log.txt", true); sw.WriteLine("New entry");
```

Learning points:

Problem 7

Check if a file exists before reading it.

```
if (File.Exists(path)) { string c = File.ReadAllText(path); }
```

Expected output:

```
if (File.Exists(path)) { string c = File.ReadAllText(path); }
```

Learning points:

Problem 8

Use async to read a file asynchronously.

```
string content = await File.ReadAllTextAsync("data.txt");
```

Expected output:

```
string content = await File.ReadAllTextAsync("data.txt");
```

Learning points:

Problem 9

Write binary data using BinaryWriter.

```
using var bw = new BinaryWriter(File.OpenWrite("data.bin")); bw.Write(42);
```

Expected output:

```
using var bw = new BinaryWriter(File.OpenWrite("data.bin")); bw.Write(42);
```

Learning points:

Problem 10

List all .txt files in a directory.

```
string[] files = Directory.GetFiles("reports", "*.txt");
```

Expected output:

```
string[] files = Directory.GetFiles("reports", "*.txt");
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between File.ReadAllText and StreamReader?

A: ReadAllText reads the entire file in one call; StreamReader is a stream-based reader that processes content incrementally, useful for large files.

Q2: When should you use async file I/O?

A: When reading/writing large files or doing I/O on UI threads to keep the application responsive.

Q3: What is the purpose of FileMode in FileStream?

A: It specifies how the file should be opened: Create, Open, OpenOrCreate, Append, or Truncate.

Q4: How do you copy a file in C#?

A: Use File.Copy(source, dest) which handles the entire copy operation.

Q5: What is the difference between BinaryWriter and StreamWriter?

A: BinaryWriter writes primitive types as raw bytes; StreamWriter writes text with encoding.

Q6: How do you delete a file?

A: Use File.Delete(path) which removes the file; check File.Exists first if needed.

Q7: What happens if you try to read a file that does not exist?

A: FileNotFoundException is thrown; check File.Exists before reading to avoid this.

Q8: What is the difference between File.Open and FileStream constructor?

A: File.Open is a static convenience method; new FileStream(path, mode) is the explicit constructor with more options.

Q9: How do you write a byte array to a file?

A: Use `File.WriteAllBytes(path, bytes)` or write via `FileStream` with a `BinaryWriter`.

Q10: What is the `using` statement's role in file I/O?

A: It ensures the stream or file handle is closed and disposed even if an exception occurs.

Topic 16: Async/Await

Async/await lets you write asynchronous code that reads like synchronous code while keeping the thread responsive. A Task represents an asynchronous operation. async marks a method as asynchronous; await pauses execution until a Task completes. CancellationToken cooperatively cancels long-running operations. Task.WhenAll and Task.WhenAny enable parallel operations.

Learn From

Study the async/await fundamentals, Task-based patterns, and CancellationToken for cooperative cancellation.

- <https://learn.microsoft.com/en-us/dotnet/csharp/async/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/async/>
- <https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtoken>
- <https://learn.microsoft.com/en-us/dotnet/csharp/async-in-aspnet-core>

Key Concepts

- async methods return Task or Task<T> (or void for event handlers)
- await suspends the method until the Task completes, then resumes
- ConfigureAwait(false) avoids capturing the synchronization context
- Task.Run offloads CPU-bound work to the thread pool
- Task.WhenAll waits for all tasks to complete
- Task.WhenAny completes when any one task finishes
- CancellationToken enables cooperative cancellation with IsCancellationRequested
- CancellationTokenSource creates and manages the token
- ValueTask is a performance-optimized alternative to Task for hot paths
- Exception handling in async uses normal try/catch around await calls

Problem 1

Write an async method that returns Task<string> with a 1-second delay.

```
async Task<string> GetDelayedAsync() { await Task.Delay(1000); return "done"; }
```

Expected output:

```
async Task<string> GetDelayedAsync() { await Task.Delay(1000); return "done"; }
```

Learning points:

Problem 2

Await a Task<int> and store the result.

```
int result = await GetNumberAsync();
```

Expected output:

```
int result = await GetNumberAsync();
```

Learning points:

Problem 3

Run two async operations in parallel using Task.WhenAll.

```
var task1 = GetAsync("a"); var task2 = GetAsync("b"); await Task.WhenAll(task1, task2);
```

Expected output:

```
var task1 = GetAsync("a"); var task2 = GetAsync("b"); await Task.WhenAll(task1, task2);
```

Learning points:

Problem 4

Cancel an async operation using CancellationToken.

```
var cts = new CancellationTokenSource(); await LongRunningAsync(cts.Token);
```

Expected output:

```
var cts = new CancellationTokenSource(); await LongRunningAsync(cts.Token);
```

Learning points:

Problem 5

Check for cancellation inside an async loop.

```
while (!token.IsCancellationRequested) { await Task.Delay(100); }
```

Expected output:

```
while (!token.IsCancellationRequested) { await Task.Delay(100); }
```

Learning points:

Problem 6

Use Task.Run to execute CPU-bound work asynchronously.

```
int result = await Task.Run(() => HeavyComputation());
```

Expected output:

```
int result = await Task.Run(() => HeavyComputation());
```

Learning points:

Problem 7

Use ConfigureAwait(false) to avoid deadlocks in library code.

```
await GetDataAsync().ConfigureAwait(false);
```

Expected output:

```
await GetDataAsync().ConfigureAwait(false);
```

Learning points:

Problem 8

Handle exceptions from an async method with try/catch.

```
try { await RiskyAsync(); } catch (Exception ex) { Console.WriteLine(ex.Message); }
```

Expected output:

```
try { await RiskyAsync(); } catch (Exception ex) { Console.WriteLine(ex.Message); }
```

Learning points:

Problem 9

Cancel a Task after a timeout using CancellationTokenSource.

```
var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5)); await LongOpAsync(cts.Token);
```

Expected output:

```
var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5)); await LongOpAsync(cts.Token);
```

Learning points:

Problem 10

Use Task.WhenAny to get the first completed result from multiple sources.

```
var winner = await Task.WhenAny(task1, task2); var result = await winner;
```

Expected output:

```
var winner = await Task.WhenAny(task1, task2); var result = await winner;
```

Learning points:

Quick Questions & Answers

Q1: What happens when you call an async method without await?

A: The method starts executing synchronously until the first await, then returns an incomplete Task; the caller gets the Task but may not wait for completion.

Q2: What is the difference between Task and ValueTask?

A: Task is a reference type that allocates on the heap; ValueTask is a struct optimized for scenarios where the result is often available synchronously.

Q3: Why use ConfigureAwait(false)?

A: It prevents capturing the SynchronizationContext, avoiding deadlocks in library code that does not need to resume on the original thread.

Q4: Can you await multiple tasks in parallel?

A: Yes; Task.WhenAll waits for all tasks; Task.WhenAny completes when any one finishes.

Q5: What is CancellationTokenSource?

A: It creates a CancellationToken and provides Cancel() to signal cancellation; it can also accept a timeout.

Q6: What is the difference between Task.Run and Task.Factory.StartNew?

A: Task.Run is a simpler, recommended API for offloading work; Task.Factory.StartNew offers more control over scheduling and options.

Q7: How do you handle exceptions from multiple parallel tasks?

A: Task.WhenAll throws AggregateException; use try/catch and inspect the InnerExceptions collection.

Q8: What is the async void problem?

A: async void methods cannot be awaited and exceptions cannot be caught; use async Task except for event handlers.

Q9: What is the synchronization context?

A: It determines which thread the continuation runs on; in UI apps it posts to the UI thread; ConfigureAwait(false) skips this.

Q10: How do you cancel a running Task?

A: Pass a CancellationToken to the async method; check IsCancellationRequested or call token.ThrowIfCancellationRequested().

Topic 17: Regex in C#

The `System.Text.RegularExpressions.Regex` class provides powerful pattern matching and text manipulation. You define patterns with metacharacters, use `Match` and `Matches` to find results, and extract data with groups. Regex is useful for validation, parsing, and text transformation.

Learn From

Study the `Regex` class API and common patterns for emails, URLs, and numeric validation.

- <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions>
- <https://learn.microsoft.com/en-us/dotnet/api/system.text.regularexpressions.regex>
- <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expression-language-quick-reference>
- <https://learn.microsoft.com/en-us/dotnet/standard/base-types/common-techniques>

Key Concepts

- `Regex.Match` returns the first match; `Regex.Matches` returns all matches
- Groups capture substrings matched by parenthesized parts of the pattern
- Named groups (`?<name>...`) let you extract values by name
- Quantifiers: `*` (zero or more), `+` (one or more), `?` (zero or one), `{n,m}`
- Character classes: `\d` (digit), `\w` (word char), `\s` (whitespace), `[abc]`
- Anchors: `^` (start of string), `$` (end of string), `\b` (word boundary)
- `RegexOptions.IgnoreCase` and `RegexOptions.Multiline` modify behavior
- `Regex.Replace` substitutes matched patterns with new text
- `Regex.IsMatch` checks if a pattern matches (bool)
- Compiled option improves performance for frequently used patterns

Problem 1

Check if a string matches the pattern for a valid email address.

```
bool valid = Regex.IsMatch(email, @"^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$");
```

Expected output:

```
bool valid = Regex.IsMatch(email, @"^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$");
```

Learning points:

Problem 2

Extract all digits from a string using `Regex.Matches`.

```
MatchCollection matches = Regex.Matches(input, @"\d+");
```

Expected output:

```
MatchCollection matches = Regex.Matches(input, @"\d+");
```

Learning points:

Problem 3

Use a named group to extract a date part from a string.

```
var match = Regex.Match(input, @"(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})"); string year = match.Groups["year"];
```

Expected output:

```
var match = Regex.Match(input, @"(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})"); string year = match.Groups["year"];
```

Learning points:

Problem 4

Replace all spaces in a string with hyphens using `Regex.Replace`.

```
string result = Regex.Replace(input, @"\s+", "-");
```

Expected output:

```
string result = Regex.Replace(input, @"\s+", "-");
```

Learning points:

Problem 5

Check if a string is a valid US phone number pattern (123-456-7890).

```
bool valid = Regex.IsMatch(phone, @"^\d{3}-\d{3}-\d{4}$");
```

Expected output:

```
bool valid = Regex.IsMatch(phone, @"^\d{3}-\d{3}-\d{4}$");
```

Learning points:

Problem 6

Find all URLs in a string using a regex pattern.

```
var urls = Regex.Matches(text, @"https?:\/\/[^\s]+", RegexOptions.IgnoreCase);
```

Expected output:

```
var urls = Regex.Matches(text, @"https?:\/\/[^\s]+", RegexOptions.IgnoreCase);
```

Learning points:

Problem 7

Use `Regex.Split` to split a string on commas or semicolons.

```
string[] parts = Regex.Split(input, @"[,;]+");
```

Expected output:

```
string[] parts = Regex.Split(input, @"[,;]+");
```

Learning points:

Problem 8

Match a string that starts with a capital letter followed by lowercase letters.

```
bool match = Regex.IsMatch(input, @"^[A-Z][a-z]+$");
```

Expected output:

```
bool match = Regex.IsMatch(input, @"^[A-Z][a-z]+$");
```

Learning points:

Problem 9

Extract the first group value from a match.

```
string val = Regex.Match(input, @"(\d+)").Groups[1].Value;
```

Expected output:

```
string val = Regex.Match(input, @"(\d+)").Groups[1].Value;
```

Learning points:

Problem 10

Use Regex with IgnoreCase and Multiline options.

```
var match = Regex.Match(input, @"^hello$", RegexOptions.IgnoreCase | RegexOptions.Multiline);
```

Expected output:

```
var match = Regex.Match(input, @"^hello$", RegexOptions.IgnoreCase | RegexOptions.Multiline);
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between Match and Matches?

A: Match returns a single Match object for the first match; Matches returns a MatchCollection containing all non-overlapping matches.

Q2: How do named groups work?

A: The syntax (?<name>pattern) creates a named group; access its value with match.Groups["name"].Value.

Q3: What is the difference between \d and [0-9]?

A: They are equivalent; \d is a shorthand character class for digits 0-9.

Q4: When should you use RegexOptions.Compiled?

A: When the regex is used frequently and performance is critical; it pre-compiles the pattern to IL for faster execution.

Q5: What is the difference between Regex.IsMatch and Match.Success?

A: Regex.IsMatch is a static method that returns bool; Match.Success is a property on a Match object indicating whether a match was found.

Q6: How do you escape special characters in a regex?

A: Use \ before special characters like . * + ? ^ \$ () [] { } |, or use Regex.Escape().

Q7: What does the ? quantifier mean?

A: It makes the preceding element optional (zero or one occurrence).

Q8: What is a lazy vs greedy quantifier?

A: Greedy quantifiers (* + ?) match as much as possible; lazy quantifiers (*? +? ??) match as little as possible.

Q9: What does \b match?

A: A word boundary—the position between a word character and a non-word character.

Q10: Can you use regex with async code?

A: Regex operations are synchronous and CPU-bound; wrap them in Task.Run for async contexts.

Topic 18: NuGet & Packaging

NuGet is the .NET package manager. It provides a repository of libraries that you can add to your projects. You can create your own NuGet packages to share code with others. Understanding NuGet is essential for managing dependencies and distributing your libraries.

Learn From

Study NuGet package creation, the dotnet CLI for packaging, and the NuGet.org publishing workflow.

- <https://learn.microsoft.com/en-us/nuget/>
- <https://learn.microsoft.com/en-us/nuget/create-packages/creating-a-package>
- <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-pack>
- <https://learn.microsoft.com/en-us/nuget/quickstart/create-and-publish-a-package>

Key Concepts

- `dotnet add package` adds a NuGet dependency to a project
- NuGet packages are .nupkg files containing assemblies and metadata
- The .nuspec or .csproj file defines package metadata (version, authors, description)
- PackageReference is the modern way to reference NuGet packages in .csproj
- `dotnet pack` creates a .nupkg from a project
- `dotnet nuget push` publishes a package to NuGet.org or a private feed
- Semantic versioning (SemVer) defines version numbering: MAJOR.MINOR.PATCH
- Package dependencies are resolved automatically by NuGet
- Private NuGet feeds can be configured in nuget.config
- Source Link embeds source code links in packages for debugging

Problem 1

Add the Newtonsoft.Json package to a project using dotnet CLI.

```
dotnet add package Newtonsoft.Json
```

Expected output:

```
dotnet add package Newtonsoft.Json
```

Learning points:

Problem 2

Create a NuGet package from a project.

```
dotnet pack
```

Expected output:

```
dotnet pack
```

Learning points:

Problem 3

Publish a NuGet package to NuGet.org.

```
dotnet nuget push *.nupkg --api-key <API_KEY> --source https://api.nuget.org/v3/index.json
```

Expected output:

```
dotnet nuget push *.nupkg --api-key <API_KEY> --source https://api.nuget.org/v3/index.json
```

Learning points:

Problem 4

Specify a package version in the .csproj file.

```
<Version>1.0.0</Version>
```

Expected output:

```
<Version>1.0.0</Version>
```

Learning points:

Problem 5

Add a package dependency with a version range.

```
<PackageReference Include="Newtonsoft.Json" Version="[13.0.1, 14.0.0)" />
```

Expected output:

```
<PackageReference Include="Newtonsoft.Json" Version="[13.0.1, 14.0.0)" />
```

Learning points:

Problem 6

List installed NuGet packages in a project.

```
dotnet list package
```

Expected output:

```
dotnet list package
```

Learning points:

Problem 7

Restore NuGet packages for a solution.

```
dotnet restore
```

Expected output:

```
dotnet restore
```

Learning points:

Problem 8

Create a NuGet package with a specific output directory.

```
dotnet pack -o ./nupkgs
```

Expected output:

```
dotnet pack -o ./nupkgs
```

Learning points:

Problem 9

Add a private NuGet source.

```
dotnet nuget add source https://myrepo.com/v3/index.json --name MyFeed
```

Expected output:

```
dotnet nuget add source https://myrepo.com/v3/index.json --name MyFeed
```

Learning points:

Problem 10

Update a specific NuGet package to the latest version.

```
dotnet add package Newtonsoft.Json --latest
```

Expected output:

```
dotnet add package Newtonsoft.Json --latest
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between PackageReference and packages.config?

A: PackageReference is the modern format stored in .csproj; packages.config is the legacy format with an XML file listing packages.

Q2: What is semantic versioning?

A: A versioning scheme using MAJOR.MINOR.PATCH where MAJOR has breaking changes, MINOR adds features, PATCH fixes bugs.

Q3: How do you create a NuGet package from a class library?

A: Run dotnet pack which generates a .nupkg containing the compiled assemblies and package metadata.

Q4: What is a NuGet source?

A: A feed URL from which NuGet resolves and downloads packages; NuGet.org is the default public source.

Q5: What is the difference between a stable and prerelease package?

A: Stable packages have no suffix; prerelease packages include -alpha, -beta, -rc suffixes.

Q6: How do you specify a package description?

A: In the .csproj file with <Description>My library description</Description>.

Q7: What is Source Link?

A: A feature that embeds source code URLs in debug symbols, enabling step-through debugging of NuGet packages.

Q8: Can you reference a local project as a NuGet package?

A: Yes; use ProjectReference for development or create a local .nupkg and reference it as a PackageReference.

Q9: What is the difference between dotnet restore and dotnet build?

A: Restore downloads package dependencies; build compiles the code. Build implicitly runs restore if needed.

Q10: How do you uninstall a NuGet package?

A: Run dotnet remove package <PackageName> which removes the PackageReference from the .csproj.

Topic 19: ASP.NET Basics

ASP.NET Core is the cross-platform web framework for building APIs and web apps. Minimal APIs provide a lightweight way to define endpoints with minimal ceremony. Controllers offer the traditional MVC pattern. Middleware pipeline processes requests. Routing maps URLs to handlers.

Learn From

Start with the ASP.NET Core getting-started guide, then explore minimal APIs and middleware.

- <https://learn.microsoft.com/en-us/aspnet/core/getting-started/>
- <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis>
- <https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/actions>
- <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware>

Key Concepts

- `WebApplication.CreateBuilder` creates a configurable host
- `app.MapGet/MapPost/MapPut/MapDelete` define minimal API endpoints
- Controllers use `[Route]`, `[HttpGet]`, `[HttpPost]` attributes for routing
- Middleware components process requests in a pipeline
- `app.Use*` adds middleware; `app.Map*` branches the pipeline
- Dependency injection (DI) registers services in the container
- `app.Run()` starts the web server
- Request and Response objects provide HTTP data
- Model binding maps request data to method parameters
- Filters intercept controller actions for cross-cutting concerns

Problem 1

Create a minimal API that responds GET to /hello with "Hello, World!".

```
var app = WebApplication.CreateBuilder(args).Build(); app.MapGet("/hello", () => "Hello, World!"); app.Run()
```

Expected output:

```
var app = WebApplication.CreateBuilder(args).Build(); app.MapGet("/hello", () => "Hello, World!"); app.Run();
```

Learning points:

Problem 2

Create a minimal API endpoint that accepts a name parameter.

```
app.MapGet("/greet/{name}", (string name) => $"Hello, {name}!");
```

Expected output:

```
app.MapGet("/greet/{name}", (string name) => $"Hello, {name}!");
```

Learning points:

Problem 3

Create a minimal API POST endpoint that accepts a JSON body.

```
app.MapPost("/items", (Item item) => Results.Created($"/items/1", item));
```

Expected output:

```
app.MapPost("/items", (Item item) => Results.Created($"/items/1", item));
```

Learning points:

Problem 4

Configure a service in the DI container.

```
builder.Services.AddSingleton<IMyService, MyService>();
```

Expected output:

```
builder.Services.AddSingleton<IMyService, MyService>();
```

Learning points:

Problem 5

Add a middleware that logs the request path.

```
app.Use(async (context, next) => { Console.WriteLine(context.Request.Path); await next(); });
```

Expected output:

```
app.Use(async (context, next) => { Console.WriteLine(context.Request.Path); await next(); });
```

Learning points:

Problem 6

Create a controller with a `[Route("api/[controller]")]` attribute.

```
[Route("api/[controller]")] public class UsersController : Controller { }
```

Expected output:

```
[Route("api/[controller]")] public class UsersController : Controller { }
```

Learning points:

Problem 7

Map a route to a controller action using `[HttpGet("{id}")]`.

```
[HttpGet("{id}")] public IActionResult Get(int id) => Ok(id);
```

Expected output:

```
[HttpGet("{id}")] public IActionResult Get(int id) => Ok(id);
```

Learning points:

Problem 8

Add CORS middleware to allow all origins.

```
app.UseCors(policy => policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
```

Expected output:

```
app.UseCors(policy => policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
```

Learning points:

Problem 9

Use `app.UseExceptionHandler` to add global error handling.

```
app.UseExceptionHandler(error => { error.Run(async context => { context.Response.StatusCode = 500; }); });
```

Expected output:

```
app.UseExceptionHandler(error => { error.Run(async context => { context.Response.StatusCode = 500; }); });
```

Learning points:

Problem 10

Return a 404 Not Found from a minimal API endpoint.

```
app.MapGet("/check", () => Results.NotFound());
```

Expected output:

```
app.MapGet("/check", () => Results.NotFound());
```

Learning points:

Quick Questions & Answers

Q1: What is the difference between minimal APIs and controllers?

A: Minimal APIs are lightweight, attribute-free, and ideal for simple APIs; controllers use attributes and support features like model validation and filters.

Q2: How does middleware pipeline work?

A: Each middleware component calls `next()` to pass the request to the next component; if not called, the pipeline short-circuits.

Q3: What is dependency injection in ASP.NET?

A: The framework creates and injects service instances into constructors, promoting loose coupling and testability.

Q4: How do you configure Kestrel to listen on a specific port?

A: Use `builder.WebHost.ConfigureKestrel(options => options.ListenLocalhost(5000))` or use `appsettings.json`.

Q5: What is model binding?

A: The process of mapping request data (query strings, route data, body) to method parameters or model objects.

Q6: How do you return JSON from a minimal API?

A: Return an object directly; ASP.NET serializes it to JSON automatically with the default JSON serializer.

Q7: What is the difference between `app.Use` and `app.Map`?

A: `Use` adds middleware to the pipeline; `Map` branches the pipeline based on a path prefix.

Q8: How do you add authentication middleware?

A: Use `app.UseAuthentication()` and `app.UseAuthorization()` after adding services with `AddAuthentication()`.

Q9: What is the Purpose of [FromBody] and [FromQuery] attributes?

A: They specify the source of model binding: `FromBody` reads from the request body; `FromQuery` reads from query string parameters.

Q10: Can minimal APIs use filters?

A: Yes; since .NET 7, minimal APIs support endpoint filters for validation and cross-cutting concerns.

Topic 20: .NET Reflection

Reflection lets you inspect and invoke types, methods, properties, and fields at runtime. The `System.Reflection` namespace provides `Type`, `MethodInfo`, `PropertyInfo`, and `FieldInfo` classes. Reflection enables dynamic loading, late binding, and metadata inspection, but incurs a performance cost.

Learn From

Study the `System.Reflection` namespace and practice inspecting types and invoking methods dynamically.

- <https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection-and-loading>
- <https://learn.microsoft.com/en-us/dotnet/api/system.type>
- <https://learn.microsoft.com/en-us/dotnet/api/system.reflection.methodinfo>
- <https://learn.microsoft.com/en-us/dotnet/api/system.reflection.propertyinfo>

Key Concepts

- `typeof(T)` and `obj.GetType()` obtain the `Type` of a class
- `Type.GetMethods()` returns all methods; `GetProperties()` returns all properties
- `MethodInfo.Invoke()` dynamically calls a method by reflection
- `PropertyInfo.GetValue` and `SetValue` read/write properties dynamically
- `FieldInfo` reads/writes fields at runtime
- `Assembly.Load` loads assemblies dynamically from bytes or files
- `CustomAttributes` provides access to attributes on types and members
- `BindingFlags` control which members are retrieved (public, static, instance)
- `Activator.CreateInstance` creates instances of types dynamically
- Reflection is used by serialization, ORMs, DI containers, and testing frameworks

Problem 1

Get the `Type` of a string variable using reflection.

```
Type t = typeof(string);
```

Expected output:

```
Type t = typeof(string);
```

Learning points:

Problem 2

Get all public methods of a type.

```
MethodInfo[] methods = typeof(MyClass).GetMethods(BindingFlags.Public | BindingFlags.Instance);
```

Expected output:

```
MethodInfo[] methods = typeof(MyClass).GetMethods(BindingFlags.Public | BindingFlags.Instance);
```

Learning points:

Problem 3

Invoke a method by name using MethodInfo.Invoke.

```
MethodInfo m = typeof(MyClass).GetMethod("DoWork"); m.Invoke(obj, new object[] { });
```

Expected output:

```
MethodInfo m = typeof(MyClass).GetMethod("DoWork"); m.Invoke(obj, new object[] { });
```

Learning points:

Problem 4

Get a property value using PropertyInfo.GetValue.

```
PropertyInfo p = typeof(MyClass).GetProperty("Name"); string val = (string)p.GetValue(obj);
```

Expected output:

```
PropertyInfo p = typeof(MyClass).GetProperty("Name"); string val = (string)p.GetValue(obj);
```

Learning points:

Problem 5

Create an instance of a type dynamically using Activator.

```
object instance = Activator.CreateInstance(typeof(MyClass));
```

Expected output:

```
object instance = Activator.CreateInstance(typeof(MyClass));
```

Learning points:

Problem 6

Check if a type implements an interface.

```
bool implements = typeof(MyClass).IsAssignableFrom(typeof(IMyInterface));
```

Expected output:

```
bool implements = typeof(MyClass).IsAssignableFrom(typeof(IMyInterface));
```

Learning points:

Problem 7

Get custom attributes on a class.

```
var attrs = typeof(MyClass).GetCustomAttributes(typeof(ObsoleteAttribute), false);
```

Expected output:

```
var attrs = typeof(MyClass).GetCustomAttributes(typeof(ObsoleteAttribute), false);
```

Learning points:

Problem 8

Set a property value dynamically with `PropertyInfo.SetValue`.

```
PropertyInfo p = typeof(MyClass).GetProperty("Count"); p.SetValue(obj, 42);
```

Expected output:

```
PropertyInfo p = typeof(MyClass).GetProperty("Count"); p.SetValue(obj, 42);
```

Learning points:

Problem 9

Get all properties of a type that are strings.

```
var strProps = typeof(MyClass).GetProperties().Where(p => p.PropertyType == typeof(string));
```

Expected output:

```
var strProps = typeof(MyClass).GetProperties().Where(p => p.PropertyType == typeof(string));
```

Learning points:

Problem 10

Use `Type.GetConstructor` to create an instance with constructor parameters.

```
var ctor = typeof(MyClass).GetConstructor(new[] { typeof(string) }); object obj = ctor.Invoke(new object[] {
```

Expected output:

```
var ctor = typeof(MyClass).GetConstructor(new[] { typeof(string) }); object obj = ctor.Invoke(new object[] { "test"
```

Learning points:

Quick Questions & Answers

Q1: What is the performance cost of reflection?

A: Reflection is significantly slower than direct calls due to dynamic type checking and boxing; cache reflected members for repeated use.

Q2: What are BindingFlags?

A: Flags that control which members are retrieved: Public, Private, Static, Instance, DeclaredOnly, FlattenHierarchy.

Q3: What is the difference between typeof and GetType?

A: `typeof(T)` is a compile-time operation; `GetType()` is a runtime method on an object instance.

Q4: How does Activator.CreateInstance work?

A: It dynamically creates an instance of a type using its constructor; you can pass constructor parameters as arguments.

Q5: What is the difference between Property and Field info?

A: `PropertyInfo` wraps properties (get/set accessors); `FieldInfo` wraps fields directly.

Q6: How do you get generic type arguments?

A: Use `Type.GetGenericArguments()` which returns an array of `Type` objects for the type parameters.

Q7: What is dynamic invocation?

A: Calling a method or accessing a property by name at runtime using reflection or the `dynamic` keyword.

Q8: What is Assembly.Load used for?

A: To dynamically load an assembly from a file or byte array, enabling plugin architectures.

Q9: Can reflection access private members?

A: Yes; using `BindingFlags.NonPublic`, reflection can access private fields, methods, and properties.

Q10: What are the alternatives to reflection?

A: Source generators, expression trees, dynamic keyword, and compiled delegates offer better performance for known types.

Topic 21: IL (Intermediate Language)

Intermediate Language (IL) is the low-level instruction set that C# compiles to before the JIT converts it to native code. Understanding IL helps you understand how language constructs work under the hood and is essential for decompilation and security analysis.

Learn From

Study IL opcodes with the Common Intermediate Language reference and use ildasm to inspect assemblies.

- <https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit>
- <https://learn.microsoft.com/en-us/dotnet/framework/tools/ildasm-exe>
- <https://learn.microsoft.com/en-us/dotnet/standard/assembly/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/stackalloc>

Key Concepts

- IL is a stack-based instruction set; operations push and pop values from an evaluation stack
- Common opcodes: ldloc (load local), stloc (store local), ldc (load constant), call, ret
- The JIT compiler converts IL to native machine code at runtime
- Metadata in assemblies describes types, methods, fields, and references
- ildasm is the IL Disassembler that displays assembly contents as IL
- Assembly contains a manifest, type definitions, and IL code
- Value types are boxed/unboxed when moving to/from the IL stack
- Exception handling in IL uses try/catch/filter blocks
- The CLR manages memory, type safety, and security through IL verification
- Understanding IL helps identify obfuscation and reverse-engineering techniques

Problem 1

Use ildasm to view the IL of a compiled assembly.

```
ildasm MyApp.dll
```

Expected output:

```
ildasm MyApp.dll
```

Learning points:

Problem 2

Identify the IL opcode for loading an integer constant onto the stack.

```
ldc.i4 <value>
```

Expected output:

```
ldc.i4 <value>
```

Learning points:

Problem 3

What IL opcode is used to call a static method?

```
call
```

Expected output:

```
call
```

Learning points:

Problem 4

What IL opcode returns from a method?

```
ret
```

Expected output:

```
ret
```

Learning points:

Problem 5

What is the IL for a simple if/else statement?

```
brfalse/brtrue to jump past the false branch, then br to skip the true branch
```

Expected output:

```
brfalse/brtrue to jump past the false branch, then br to skip the true branch
```

Learning points:

Problem 6

What IL opcode stores a value into a local variable?

```
stloc
```

Expected output:

```
stloc
```

Learning points:

Problem 7

What is a metadata token in IL?

```
A numeric identifier that references a type, method, field, or string in the assembly metadata
```

Expected output:

```
A numeric identifier that references a type, method, field, or string in the assembly metadata
```

Learning points:

Problem 8

What is the difference between box and unbox IL opcodes?

```
box converts a value type to an object reference on the heap; unbox extracts a value type from an object
```

Expected output:

box converts a value type to an object reference on the heap; unbox extracts a value type from an object

Learning points:

Problem 9

How does the CLR verify IL for type safety?

```
The verifier checks that operations are type-safe, stack is balanced, and no invalid casts occur before JIT compilation
```

Expected output:

The verifier checks that operations are type-safe, stack is balanced, and no invalid casts occur before JIT compilation

Learning points:

Problem 10

What is the purpose of the .locals directive in IL?

```
Declares local variables for a method, specifying their types and init flag
```

Expected output:

Declares local variables for a method, specifying their types and init flag

Learning points:

Quick Questions & Answers

Q1: Why should a security researcher understand IL?

A: IL reveals the true behavior of compiled code, enabling detection of hidden logic, backdoors, and obfuscation techniques.

Q2: What is the difference between JIT and AOT compilation?

A: JIT compiles IL to native code at runtime; AOT (Ahead-of-Time) compiles to native code before deployment, reducing startup time.

Q3: How does IL enforce type safety?

A: The CLR verifier checks IL operations at load time to ensure no type violations or memory corruption.

Q4: What is a metadata token?

A: A 32-bit value that references a type, method, field, or string in the assembly's metadata tables.

Q5: What is the difference between call and callvirt IL opcodes?

A: call invokes a method directly; callvirt invokes virtually, supporting polymorphic dispatch and null checks.

Q6: How do loops translate to IL?

A: Loops use conditional branches (brtrue, brfalse) and unconditional branches (br) to create structures.

Q7: What is the purpose of the ldloc opcode?

A: It loads the value of a local variable onto the evaluation stack for use by subsequent instructions.

Q8: What is the role of the .entrypoint directive?

A: It specifies the method that serves as the assembly's entry point, typically Main.

Q9: How does exception handling work in IL?

A: IL uses .try/catch/filter directives to define exception regions, with leave instructions to transfer control.

Q10: What is the difference between valuetype and class in IL?

A: valuetype instances are stored on the stack or inline; class instances are reference types stored on the managed heap.

Topic 22: .NET Reversing with dnSpy/ILSpy

Decompilers like dnSpy and ILSpy reverse-engineer compiled .NET assemblies back into readable C# code. They are essential tools for security researchers analyzing third-party libraries, finding hardcoded secrets, understanding proprietary logic, and auditing code for vulnerabilities.

Learn From

Install dnSpy or ILSpy, open an assembly, and practice navigating the decompiled code structure.

- <https://github.com/dnSpy/dnSpy>
- <https://github.com/icsharpcode/ILSpy>
- <https://learn.microsoft.com/en-us/dotnet/standard/assembly/>
- <https://learn.microsoft.com/en-us/dotnet/framework/tools/>

Key Concepts

- dnSpy supports both decompilation and live editing/decompiling of assemblies
- ILSpy offers a lightweight, open-source alternative with plugin support
- Decompilers reconstruct C# source from IL metadata and opcodes
- Obfuscation tools like ConfuserEx and Dotfuscator make decompiled code harder to read
- Hardcoded secrets (API keys, connection strings, passwords) are visible in decompiled code
- String encryption obfuscation stores encrypted strings that are decrypted at runtime
- Method body analysis reveals algorithm logic and control flow
- Assembly references show which libraries and frameworks are used
- Metadata analysis reveals types, attributes, and versioning information
- Debug symbols (.pdb files) enable source-level debugging of decompiled code

Problem 1

Open an assembly in dnSpy and navigate to the Main method.

```
File > Open > select the assembly > expand the tree > navigate to Program.cs > Main
```

Expected output:

Navigate to Program.Main in dnSpy

Learning points:

Problem 2

Decompile a DLL and find all string literals.

```
Use Analyze > search for string references, or search for String in the search bar
```

Expected output:

Search for string references in the decompiled assembly

Learning points:

Problem 3

Identify a hardcoded API key in decompiled code.

Look for string assignments with suspicious values like "sk-..." or "api_key=..."

Expected output:

Find string constants containing API keys or secrets

Learning points:

Problem 4

Use ILSpy to view the IL of a method.

Right-click method > IL Instructions

Expected output:

View IL instructions in ILSpy

Learning points:

Problem 5

Find all methods that call a specific external method.

Use the Analyze tab > References to find callers

Expected output:

Find references to the target method

Learning points:

Problem 6

Search for all types in an assembly that implement an interface.

Use the search bar with IInterfaceName

Expected output:

Find implementing classes via search

Learning points:

Problem 7

Decompile and find the assembly version.

Check Assembly Properties or the AssemblyInfo attribute

Expected output:

Read the AssemblyVersion attribute

Learning points:

Problem 8

Identify obfuscated class names in a decompiled assembly.

```
Look for classes with names like <Module>, <>c, or random strings
```

Expected output:

Identify compiler-generated or obfuscated types

Learning points:

Problem 9

Find all configuration strings (connection strings, keys).

```
Search for strings containing "Server=", "Password=", or "key="
```

Expected output:

Search for configuration-related string literals

Learning points:

Problem 10

Export decompiled source code from ILSpy.

```
File > Save Code > select output directory
```

Expected output:

Export decompiled source to files

Learning points:

Quick Questions & Answers

Q1: What is the difference between dnSpy and ILSpy?

A: dnSpy includes a debugger and editor; ILSpy is lighter and open-source with plugin support. Both decompile .NET assemblies.

Q2: How do decompilers reconstruct C# code?

A: They read IL opcodes and metadata, then apply heuristics to reconstruct high-level constructs like classes, methods, and properties.

Q3: What is string encryption obfuscation?

A: A technique where string literals are encrypted and decrypted at runtime, making static analysis harder.

Q4: How do you find hardcoded secrets?

A: Search for string literals in decompiled code; look for patterns like API keys, passwords, and connection strings.

Q5: What are debug symbols (.pdb files)?

A: Files that map IL instructions to source code lines, enabling step-through debugging of decompiled assemblies.

Q6: How do you handle obfuscated code?

A: Use deobfuscators like de4dot, or analyze IL directly to understand the true behavior.

Q7: What information can you extract from assembly metadata?

A: Version numbers, dependencies, type definitions, custom attributes, and security permissions.

Q8: Can you modify and recompile a decompiled assembly?

A: Yes; dnSpy supports editing and recompiling method bodies directly.

Q9: What is the difference between IL and decompiled C#?

A: IL is the low-level instruction set; decompiled C# is a reconstruction that approximates the original source code.

Q10: How do you analyze a .NET assembly without decompiling?

A: Use ildasm for raw IL, monodis for Mono assemblies, or dotPeek for quick decompilation.

Topic 23: Secure Coding in C#

Secure coding prevents vulnerabilities like SQL injection, XSS, and secrets exposure. Parameterized queries prevent SQL injection. Input validation rejects malicious data. The Secret Manager and Azure Key Vault handle secrets securely. HTTPS, CORS, and authentication protect web applications.

Learn From

Study OWASP Top 10, parameterized queries, and .NET security best practices.

- <https://learn.microsoft.com/en-us/dotnet/standard/security/>
- <https://learn.microsoft.com/en-us/aspnet/core/security/>
- <https://learn.microsoft.com/en-us/dotnet/core/app-secrets>
- <https://learn.microsoft.com/en-us/azure/key-vault/>

Key Concepts

- Always use parameterized queries to prevent SQL injection
- Validate and sanitize all user input on the server side
- Use built-in validation attributes ([Required], [StringLength], [Range])
- Store secrets in Secret Manager, environment variables, or Key Vault, never in code
- Enable HTTPS and HSTS for all web applications
- Use Content Security Policy (CSP) to prevent XSS
- Hash passwords with PBKDF2, bcrypt, or Argon2, never MD5/SHA1
- Apply the principle of least privilege for database and file access
- Use anti-forgery tokens to prevent CSRF attacks
- Log security events without exposing sensitive data

Problem 1

Write a parameterized SQL query to prevent injection.

```
cmd.Parameters.AddWithValue("@name", inputName);
```

Expected output:

```
cmd.Parameters.AddWithValue("@name", inputName);
```

Learning points:

Problem 2

Validate an email input using [EmailAddress] attribute.

```
[EmailAddress] public string Email { get; set; }
```

Expected output:

```
[EmailAddress] public string Email { get; set; }
```

Learning points:

Problem 3

Store a secret using dotnet user-secrets.

```
dotnet user-secrets set "ApiKey" "mykey123"
```

Expected output:

```
dotnet user-secrets set "ApiKey" "mykey123"
```

Learning points:

Problem 4

Read a secret from configuration.

```
string key = config["ApiKey"];
```

Expected output:

```
string key = config["ApiKey"];
```

Learning points:

Problem 5

Hash a password with BCrypt.

```
string hash = BCrypt.Net.BCrypt.HashPassword(password);
```

Expected output:

```
string hash = BCrypt.Net.BCrypt.HashPassword(password);
```

Learning points:

Problem 6

Validate input length using [StringLength].

```
[StringLength(100, MinimumLength = 3)] public string Name { get; set; }
```

Expected output:

```
[StringLength(100, MinimumLength = 3)] public string Name { get; set; }
```

Learning points:

Problem 7

Use [Range] to validate a numeric input.

```
[Range(1, 100)] public int Quantity { get; set; }
```

Expected output:

```
[Range(1, 100)] public int Quantity { get; set; }
```

Learning points:

Problem 8

Enable CORS with a specific allowed origin.

```
builder.Services.AddCors(options => options.AddPolicy("Policy", policy => policy.WithOrigins("https://example.com"))
```

Expected output:

```
builder.Services.AddCors(options => options.AddPolicy("Policy", policy => policy.WithOrigins("https://example.com"))
```

Learning points:

Problem 9

Use [Authorize] to protect an API endpoint.

```
[Authorize] public IActionResult Secret() => Ok("secret");
```

Expected output:

```
[Authorize] public IActionResult Secret() => Ok("secret");
```

Learning points:

Problem 10

Sanitize HTML input to prevent XSS.

```
string safe = WebUtility.HtmlEncode(input);
```

Expected output:

```
string safe = WebUtility.HtmlEncode(input);
```

Learning points:

Quick Questions & Answers

Q1: What is SQL injection and how do you prevent it?

A: SQL injection inserts malicious SQL via user input; prevent it with parameterized queries, stored procedures, and ORM frameworks.

Q2: Why is MD5 not recommended for password hashing?

A: MD5 is fast and vulnerable to rainbow table attacks; use PBKDF2, bcrypt, or Argon2 which are slow and salted.

Q3: What is the Secret Manager tool?

A: A development tool that stores sensitive data in a local JSON file outside the project, avoiding secrets in source control.

Q4: How does input validation prevent attacks?

A: It rejects malformed or malicious input before it reaches business logic, preventing injection and buffer overflow attacks.

Q5: What is CSRF and how do you prevent it?

A: CSRF tricks users into submitting unwanted requests; prevent it with anti-forgery tokens and SameSite cookies.

Q6: What is Content Security Policy (CSP)?

A: An HTTP header that restricts which resources (scripts, styles, images) can be loaded, preventing XSS.

Q7: Why should you never store secrets in source code?

A: They become visible in version control, log files, and decompiled assemblies; use secure storage instead.

Q8: What is the principle of least privilege?

A: Grant only the minimum permissions needed to perform a task, reducing the impact of a compromise.

Q9: How do you secure API communication?

A: Use HTTPS with TLS, validate JWT tokens, and implement proper authentication and authorization.

Q10: What is the difference between authentication and authorization?

A: Authentication verifies identity; authorization determines what an authenticated user is allowed to do.

Topic 24: Memory & Performance

Understanding memory management in .NET helps you write performant applications. `Span<T>` provides a type-safe view over contiguous memory without allocation. `stackalloc` allocates on the stack for short-lived buffers. `ArrayPool<T>` reuses arrays to reduce GC pressure. `BenchmarkDotNet` measures performance accurately.

Learn From

Study `Span<T>`, `stackalloc`, `ArrayPool`, and `BenchmarkDotNet` for performance optimization.

- <https://learn.microsoft.com/en-us/dotnet/api/system.span-1>
- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/stackalloc>
- <https://learn.microsoft.com/en-us/dotnet/api/system.buffers.arraypool-1>
- <https://benchmarkdotnet.org/>

Key Concepts

- `Span<T>` is a ref struct that represents a contiguous region of memory
- `stackalloc` allocates memory on the stack instead of the heap
- `ArrayPool<T>` provides reusable array buffers to minimize allocations
- `BenchmarkDotNet` provides accurate, statistical performance measurements
- `ValueTask` avoids `Task` allocation when the result is synchronous
- String interpolation handlers optimize string creation in .NET 6+
- `ref` returns and `ref locals` avoid copying structs
- `Memory<T>` is a heap-friendly alternative to `Span<T>` for async scenarios
- Pooling reduces GC collections and improves throughput
- Use `Array.Empty<T>()` instead of `new T[0]` to avoid allocations

Problem 1

Create a `Span<int>` from an array.

```
Span<int> span = arr.AsSpan();
```

Expected output:

```
Span<int> span = arr.AsSpan();
```

Learning points:

Problem 2

Use `stackalloc` to allocate a buffer of 128 bytes on the stack.

```
Span<byte> buffer = stackalloc byte[128];
```

Expected output:

```
Span<byte> buffer = stackalloc byte[128];
```

Learning points:

Problem 3

Get a shared array from `ArrayPool<int>`.

```
int[] array = ArrayPool<int>.Shared.Rent(1024);
```

Expected output:

```
int[] array = ArrayPool<int>.Shared.Rent(1024);
```

Learning points:

Problem 4

Return an array to the pool after use.

```
ArrayPool<int>.Shared.Return(array);
```

Expected output:

```
ArrayPool<int>.Shared.Return(array);
```

Learning points:

Problem 5

Slice a `Span` to get a sub-range.

```
Span<int> slice = span.Slice(2, 5);
```

Expected output:

```
Span<int> slice = span.Slice(2, 5);
```

Learning points:

Problem 6

Use `MemoryMarshal` to read a struct from a byte span.

```
MyStruct s = MemoryMarshal.Read<MyStruct>(byteSpan);
```

Expected output:

```
MyStruct s = MemoryMarshal.Read<MyStruct>(byteSpan);
```

Learning points:

Problem 7

Use `ref` return to avoid copying a struct.

```
public ref int GetRef() => ref _field;
```

Expected output:

```
public ref int GetRef() => ref _field;
```

Learning points:

Problem 8

Create a ValueTask that returns a cached result.

```
ValueTask<int> GetCachedAsync(int id) => new ValueTask<int>(cachedValue);
```

Expected output:

```
ValueTask<int> GetCachedAsync(int id) => new ValueTask<int>(cachedValue);
```

Learning points:

Problem 9

Use Array.Empty<T>() to avoid allocating an empty array.

```
return Array.Empty<string>();
```

Expected output:

```
return Array.Empty<string>();
```

Learning points:

Problem 10

Write a BenchmarkDotNet benchmark class.

```
[MemoryDiagnoser] public class MyBenchmark { [Benchmark] public void Method() { } }
```

Expected output:

```
[MemoryDiagnoser] public class MyBenchmark { [Benchmark] public void Method() { } }
```

Learning points:

Quick Questions & Answers

Q1: What is Span<T> and why is it important?

A: Span<T> is a ref struct that provides a type-safe view over memory without heap allocation, reducing GC pressure for buffer operations.

Q2: What are the limitations of Span<T>?

A: It is a ref struct and cannot be used as a field, in async methods, or as a generic type parameter.

Q3: When should you use stackalloc?

A: For short-lived, small buffers (typically under 1KB) where stack allocation avoids heap pressure.

Q4: What is ArrayPool<T>?

A: A pool of reusable arrays that reduces allocation and garbage collection overhead for temporary buffers.

Q5: What is ValueTask vs Task?

A: ValueTask avoids heap allocation when the result is available synchronously; use Task for methods that are always async.

Q6: How does BenchmarkDotNet work?

A: It runs methods multiple times with statistical analysis, warm-up iterations, and memory diagnostics for accurate measurements.

Q7: What is Memory<T>?

A: A heap-friendly alternative to Span<T> that can be stored in fields and used in async scenarios.

Q8: What is the [MemoryDiagnoser] attribute?

A: It tells BenchmarkDotNet to report memory allocations per operation in benchmark results.

Q9: How do you avoid boxing with structs?

A: Use generic constraints, avoid interface dispatch, and prefer ref returns to prevent value type copying.

Q10: What is string interpolation handlers in .NET 6+?

A: They optimize string interpolation by avoiding intermediate string allocations when using ISpanFormattable.

Topic 25: Capstone Projects

The capstone projects combine everything you have learned—OOP, LINQ, async, file I/O, ASP.NET, security, and performance—into practical applications. Each project targets real-world scenarios and requires integrating multiple C# concepts.

Learn From

Pick a project, plan the architecture, implement incrementally, and test thoroughly.

- <https://learn.microsoft.com/en-us/dotnet/core/project-sdk/msbuild-project-overview>
- <https://learn.microsoft.com/en-us/dotnet/core/testing/>
- <https://learn.microsoft.com/en-us/aspnet/core/test/>
- <https://learn.microsoft.com/en-us/dotnet/core/docker/>

Key Concepts

- Design before coding: define interfaces, data models, and workflows
- Use dependency injection to decouple components
- Write unit tests alongside production code
- Implement proper error handling with try/catch and logging
- Use async/await for I/O-bound operations
- Apply security best practices: input validation, parameterized queries, secrets management
- Profile and optimize performance-critical paths
- Use LINQ for data transformation and querying
- Document your code with XML comments
- Containerize your application with Docker for deployment

Problem 1

Build a CLI file organizer that sorts files by extension using File I/O and LINQ.

```
var files = Directory.GetFiles(sourceDir); var grouped = files.GroupBy(f => Path.GetExtension(f));
```

Expected output:

```
var files = Directory.GetFiles(sourceDir); var grouped = files.GroupBy(f => Path.GetExtension(f));
```

Learning points:

Problem 2

Create a REST API with ASP.NET that manages a todo list with CRUD operations.

```
app.MapGet("/todos", () => todos); app.MapPost("/todos", (Todo t) => { todos.Add(t); return Results.Created($"/todos/{t.Id}", t); })
```

Expected output:

```
app.MapGet("/todos", () => todos); app.MapPost("/todos", (Todo t) => { todos.Add(t); return Results.Created($"/todos/{t.Id}", t); })
```

Learning points:

Problem 3

Implement a background task runner using async/await and CancellationToken.

```
var cts = new CancellationTokenSource(); _ = Task.Run(() => ProcessAsync(cts.Token));
```

Expected output:

```
var cts = new CancellationTokenSource(); _ = Task.Run(() => ProcessAsync(cts.Token));
```

Learning points:

Problem 4

Build a JSON configuration manager that reads/writes settings using File I/O.

```
var config = JsonSerializer.Deserialize<AppSettings>(File.ReadAllText(path));
```

Expected output:

```
var config = JsonSerializer.Deserialize<AppSettings>(File.ReadAllText(path));
```

Learning points:

Problem 5

Create a LINQ query engine that filters and sorts a CSV dataset.

```
var results = records.Where(r => r.Age > 18).OrderBy(r => r.Name).Select(r => r.ToString());
```

Expected output:

```
var results = records.Where(r => r.Age > 18).OrderBy(r => r.Name).Select(r => r.ToString());
```

Learning points:

Problem 6

Build a plugin system using reflection and interfaces.

```
Assembly.Load(pluginPath); var plugins = assembly.GetTypes().Where(t => typeof(IPlugin).IsAssignableFrom(t));
```

Expected output:

```
Assembly.Load(pluginPath); var plugins = assembly.GetTypes().Where(t => typeof(IPlugin).IsAssignableFrom(t));
```

Learning points:

Problem 7

Implement a custom logger using delegates and events.

```
public event Action<string> OnLog; void Log(string msg) => OnLog?.Invoke(msg);
```

Expected output:

```
public event Action<string> OnLog; void Log(string msg) => OnLog?.Invoke(msg);
```

Learning points:

Problem 8

Build a URL shortener using ASP.NET with in-memory storage.

```
var urls = new Dictionary<string, string>(); app.MapPost("/shorten", (string url) => { var key = Guid.NewGuid().ToString(); return key; });
```

Expected output:

```
var urls = new Dictionary<string, string>(); app.MapPost("/shorten", (string url) => { var key = Guid.NewGuid().ToString(); return key; });
```

Learning points:

Problem 9

Create a parameterized SQL data access layer using ADO.NET.

```
using var cmd = new SqlCommand("SELECT * FROM Users WHERE Id = @id", conn); cmd.Parameters.AddWithValue("@id", user.Id);
```

Expected output:

```
using var cmd = new SqlCommand("SELECT * FROM Users WHERE Id = @id", conn); cmd.Parameters.AddWithValue("@id", user.Id);
```

Learning points:

Problem 10

Build a performance-benchmarked caching layer with ArrayPool and ValueTask.

```
[MemoryDiagnoser] public class CacheBenchmark { [Benchmark] public ValueTask<int> GetOrAdd() => new ValueTask<int>(0); }
```

Expected output:

```
[MemoryDiagnoser] public class CacheBenchmark { [Benchmark] public ValueTask<int> GetOrAdd() => new ValueTask<int>(0); }
```

Learning points:

Quick Questions & Answers

Q1: How do you approach a capstone project?

A: Plan the architecture first, define interfaces, implement incrementally with tests, and refactor as you learn more.

Q2: What design patterns are useful in these projects?

A: Repository, Strategy, Observer (events), Factory, and Dependency Injection patterns are commonly used.

Q3: How do you test a REST API?

A: Use integration tests with WebApplicationFactory or HttpClient to send requests and verify responses.

Q4: What is the role of dependency injection in a capstone project?

A: It decouples components, making the code testable and maintainable by allowing interface-based programming.

Q5: How do you handle errors in a production application?

A: Use try/catch with logging, return appropriate HTTP status codes, and implement global error handling middleware.

Q6: What is the benefit of async/await in I/O-bound operations?

A: It keeps threads free for other work during I/O waits, improving scalability and responsiveness.

Q7: How do you secure a REST API?

A: Use authentication (JWT), authorization (roles/policies), input validation, and HTTPS.

Q8: What is the purpose of unit tests?

A: They verify individual components work correctly in isolation, enabling refactoring with confidence.

Q9: How do you containerize a .NET application?

A: Create a Dockerfile with the .NET SDK for building and the runtime image for execution, then build and run.

Q10: What should you consider when choosing a capstone project?

A: Choose something you are passionate about, that covers multiple topics, and that you can complete within your timeline.